

02477 Bayesian Machine Learning - Exercise 2

```
In [1]: %matplotlib inline
import pylab as plt
import pandas as pd
import jax.numpy as jnp
from jax import random
import seaborn as snb

from scipy.stats import binom as binom_dist

snb.set_style('darkgrid')
snb.set_theme(font_scale=1.)
colors = 'brgmc'
import sys
sys.path.insert(0, "../")
from TOOL_BOX_OF_FUNCTIONS.LOGISTIC_REGRESSION_WEEK_2 import LOGISTIC_REG_WEEK_2 as LR
```

Bayesian Logistic Regression for O-Ring Failures

A tidy, collapsible reference sheet

Contents

- [1. The Big Picture](#)
- [2. Model Specification](#)
- [3. Bayesian Priors](#)
- [4. Joint Distribution](#)
- [5. Posterior Predictive Distribution](#)
- [6. Quick Recap](#)

► 1. The Big Picture — Why are we doing this?

► 2. Model Specification — Likelihood, Link & Data-Generating Process

► 3. Bayesian Priors — What do we assume about α, β ?

► 4. Joint Distribution — The entire probabilistic story

► 5. Posterior Predictive Distribution — Integrating out uncertainty



Quick Recap

- Likelihood:** Binomial with a logistic link.
- Priors:** Independent Gaussians on (α, β) .
- Posterior:** Needs numerical inference.
- Posterior predictive:** Integrate predictive likelihood over the posterior to answer real-world questions about future launches.

Next step → implement in Stan / PyMC / NumPyro, run MCMC, and visualise the posterior predictive distribution.

Introduction

The purpose of this exercise is to dive deeper into the basic concepts of probabilistic and Bayesian modelling using **logistic regression** as a case-study. Specifically, we will study:

- the probability distributions involved in logistic regression
- how to approximate the posterior distribution of non-conjugate models using simple grid approximations
- how to make predictions using posterior predictive distributions

We will study these concepts using data from Challenger Space Shuttle disaster.

Content

- Part 1: Setting up the logistic regression model
- Part 2: The prior, likelihood and posterior
- Part 3: Making predictions using point estimates
- Part 4: Approximating the posterior using a grid approximation
- Part 5: Propagating uncertainty from parameters to predictions

Dataset: The Challenger Space Shuttle disaster

On a cold morning on January 28, 1986 the space shuttle called **Challenger** tragically exploded 73 seconds after its launch, killing all 7 crew members. The explosion was later determined to be caused the failure of two out of six so-called o-ring seals in one of the rocket boosters. The temperature on the day of the launch was close $0^{\circ}C$, which was record-low. The cold weather made the o-rings stiffen, reducing their ability to seal the critical joints, and hence, causing the tragic accident. O-ring failure had previously been observed on 6 of out the 23 prior launches as plotted below.

In this exercise, you will construct a simple Bayesian logistic regression model for predicting the probability of o-ring failure as a function of temperature.

The dataset $\mathcal{D} = \{(t_i, y_i)\}_{i=1}^M$ consists of $M = 23$ observations, where $y_i \in \{0, 1, \dots, 6\}$ denotes the number of failed o-rings for the i 'th temperature t_i . The number of o-rings for all launches were $N = 6$, and hence, the fraction of failed o-rings for the i 'th launch is $\frac{y_i}{N} = \frac{y_i}{6}$.

See [Wikipedia](#) for more background information.

First, we will load the data:

```
In [8]: # load data from csv file using pandas
data = pd.read_csv('./o-ring.csv')
t = data['temperature'].values
y = data['failed'].values
N = data['count'].values

data.head()
```

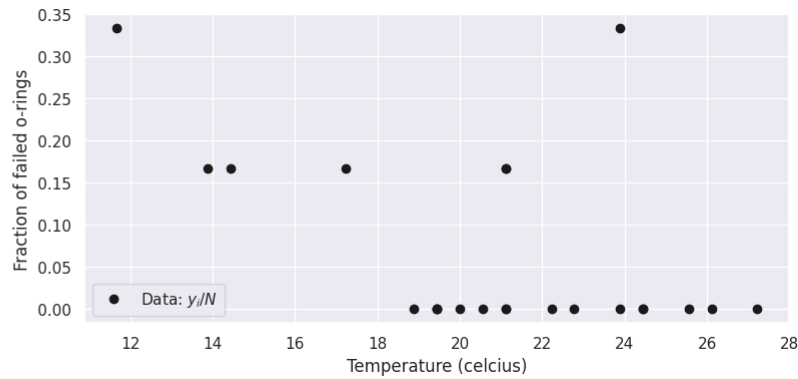
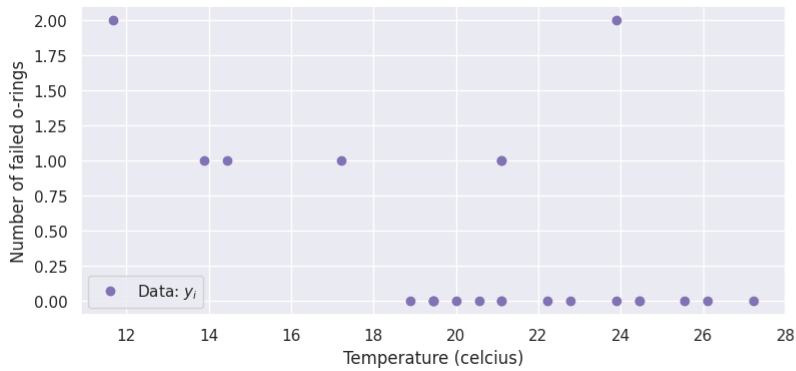
```
Out[8]:
```

	temperature	failed	count
0	18.888889	0	6
1	21.111111	1	6
2	20.555556	0	6
3	20.000000	0	6
4	19.444444	0	6

.. and plot it:

```
In [9]: # helper function for plotting data
def plot_data(ax=None, counts=True):
    if counts:
        ax.plot(t, y, 'mo', label='Data: $y_i$')
        ax.set(ylabel='Number of failed o-rings')
    else:
        ax.plot(t, y/N, 'ko', label='Data: $y_i/N$')
        ax.set(ylabel='Fraction of failed o-rings')
        ax.set(xlabel='Temperature (celcius)')
        ax.legend(loc='lower left')

# plot
fig, ax = plt.subplots(1, 2, figsize=(20, 4))
plot_data(ax[0])
plot_data(ax[1], counts=False)
```



Rather than working directly with the temperatures $t_i \in \mathbb{R}$, we will standardize the temperatures to get

$$x_i = \frac{t_i - t_{\text{mean}}}{t_{\text{std}}},$$

where t_{mean} and t_{std} are the empirical mean and standard deviation, respectively, of the observed temperatures $\{t_i\}_{i=1}^M$.

```
In [10]: # compute mean and standard deviation of temperatures
tmean, tstd = jnp.mean(t), jnp.std(t)

# standardize temperatures
standardize = lambda t_: (t_ - tmean)/tstd
x = standardize(t)
```

Part 1: Setting up the logistic regression model

The big picture

Our goal is to use the training data, i.e. the pairs of $\mathbf{x}_i$ and y_i , to learn a model such that we can make predictions and reason about the number of failures y^* for a new input point x^* . In probabilistic terms, our goal is to compute $p(y^*|\mathbf{y}, \mathbf{x}, x^*)$. That is, the distribution of the number of failures y^* **conditioned on** the observed data $\mathbf{x}, \mathbf{y}$ as well as the new input point x^* . In probabilistic machine learning, we generally refer to $p(y^*|\mathbf{y}, \mathbf{x}, x^*)$ as the **posterior predictive distribution**. Once we computed this distribution, we can use it to reason about y^* and answer questions like:

- What is the expected number of failures?
- What is the most likely number of failures?
- What is the probability that the number of failures is zero?
- What is a 90% interval credibility interval for the number of failures?

Before we can compute $p(y^*|\mathbf{y}, \mathbf{x}, x^*)$, we first need to specify a model.

Specifying the model

For each temperature x_i , we know the number of failed o-ring $y_i = \{0, 1, \dots, 6\}$ out of the total number of o-rings $N_i = 6$. Hence, the number of failing o-rings y_i for a given temperature x_i can be described by a **Binomial distribution** with $N_i = 6$ and probability $\theta_i \in [0, 1]$:

$$y_i|\theta_i \sim \text{Bin}(N_i, \theta_i). \tag{1}$$

 **Interpretation:** this is equivalent to modeling the six rings as conditionally independent Bernoulli trials with common probability θ_i for a given x_i .

This is equivalent to modelling the failure of each individual o-ring as **conditionally independent** Bernoulli trials. Since all observations have $N_i = 6$, we drop the subscript i and simply write $N = 6$ in the following to ease the notation.

Rather than imposing Beta-distributions as prior distributions for each θ_i , we assume θ_i can be modelled as is a function of the temperature. Specifically, we assume

$$\theta(x) = \sigma(\alpha + \beta x), \tag{2}$$

where $\alpha, \beta \in \mathbb{R}$ are parameters to be estimated and $\sigma: \mathbb{R} \rightarrow [0, 1]$ is the **logistic sigmoid function**, i.e. $\sigma(x) = (1 + \exp(-x))^{-1}$. For later references, we will denote the linear component of the model as $f(x)$, i.e.

$$f(x) = \alpha + \beta x. \tag{3}$$

Equivalently, the **log-odds** (logits) are linear in temperature:

$$\log\left(\frac{\theta(x)}{1-\theta(x)}\right) = f(x) = \alpha + \beta x.$$

Note that both $\theta(x)$ and $f(x)$ do indeed depend on α and β , but we suppress this dependency to ease the notation. The likelihood for a single (i 'th) data point thus becomes

$$p(y_i|x_i, \alpha, \beta) = \text{Bin}(y_i|N_i, \theta_i), \quad \text{where} \quad \theta_i = \theta(x_i) = \sigma(\alpha + \beta x_i) = \frac{1}{1 + \exp(-(\alpha + \beta x_i))}.$$

Assuming that all observations $\{y_i\}_{i=1}^M$ are **conditionally independent** given θ_i , the likelihood of the full dataset becomes

$$p(\mathbf{y}|\mathbf{x}, \alpha, \beta) = \prod_{i=1}^M p(y_i|x_i, \alpha, \beta) = \prod_{i=1}^M \text{Bin}(y_i|N, \theta_i),$$

where $\mathbf{y} = [y_1, y_2, \dots, y_M]$ and $\mathbf{x} = [x_1, x_2, \dots, x_M]$. Our goal is to be able to predict y^* for a new temperature x^* . To do this we write down the *predictive likelihood* for y^* , i.e.

$$p(y^*|x^*, \alpha, \beta) = \text{Bin}(y^*|N^*, \theta^*), \quad \text{where} \quad \theta^* = \theta(x^*) = \sigma(\alpha + \beta x^*) = \frac{1}{1 + \exp(-(\alpha + \beta x^*))}.$$

Using the **product rule** and the assumption of conditional independence, we have

$$p(\mathbf{y}, y^*|\mathbf{x}, x^*, \alpha, \beta) = p(\mathbf{y}|\mathbf{x}, \alpha, \beta) p(y^*|x^*, \alpha, \beta) = \underbrace{\prod_{i=1}^M \text{Bin}(y_i|N, \theta_i)}_{p(\mathbf{y}|\mathbf{x}, \alpha, \beta)} \underbrace{\text{Bin}(y^*|N^*, \theta^*)}_{p(y^*|x^*, \alpha, \beta)}.$$

To complete the Bayesian model, we need to impose a prior distribution on the parameters α, β . Since we do not have any information about the parameters, we will impose generic Gaussian priors with zero mean:

$$p(\alpha, \beta) = \mathcal{N}(\alpha|0, \sigma_\alpha^2) \mathcal{N}(\beta|0, \sigma_\beta^2)$$

for $\sigma_\alpha^2, \sigma_\beta^2 > 0$. This leads to the following **joint distribution** for $y^*, \mathbf{y}, \alpha$, and β :

$$p(\mathbf{y}, y^*, \alpha, \beta|\mathbf{x}, x^*) = p(y^*|x^*, \alpha, \beta) p(\mathbf{y}|\mathbf{x}, \alpha, \beta) p(\alpha, \beta) = \underbrace{\text{Bin}(y^*|N^*, \theta^*)}_{p(y^*|x^*, \alpha, \beta)} \underbrace{\prod_{i=1}^M \text{Bin}(y_i|N, \theta_i) \mathcal{N}(\alpha|0, \sigma_\alpha^2) \mathcal{N}(\beta|0, \sigma_\beta^2)}_{p(\mathbf{y}|\mathbf{x}, \alpha, \beta) p(\alpha, \beta)},$$

where we again used the product rule.

📌 **Why the joint matter:** Why the joint matters: it encodes all assumptions and dependencies. Any other quantity we care about is obtained by conditioning and/or marginalizing this joint.

The joint distribution is a central object because it completely determines how all variables (e.g. both data and parametes) interact and thus, it completely specifies our probabilistic model. From the joint distribution, we can derive any related distribution of interest using the rules of probability theory.

From the joint distribution to the posterior predictive distribution

Because we observe $\mathbf{y}$, we naturally consider the **conditional distribution** of (y^*, α, β) given the observed quantities $(\mathbf{y}, \mathbf{x}, x^*)$. Using the standard rules for conditional distributions, we get:

$$p(y^*, \alpha, \beta|\mathbf{y}, \mathbf{x}, x^*) = \frac{p(\mathbf{y}, y^*, \alpha, \beta|\mathbf{x}, x^*)}{p(\mathbf{y}|\mathbf{x})}.$$

Our primary goal is to compute the **posterior predictive distribution** $p(y^*|\mathbf{y}, \mathbf{x}, x^*)$, i.e. the **marginal distribution** of y^* conditioned on the observed data, i.e. $p(y^*|\mathbf{y}, \mathbf{x}, x^*)$ which can be obtained by marginalizing over α and β via the **sum rule**:

$$p(y^*|\mathbf{y}, \mathbf{x}, x^*) = \int \int p(y^*, \alpha, \beta|\mathbf{y}, \mathbf{x}, x^*) d\alpha d\beta = \int \int \underbrace{p(y^*|x^*, \alpha, \beta)}_{\text{likelihood for } y^*} \underbrace{p(\alpha, \beta|\mathbf{y}, \mathbf{x})}_{\text{posterior distribution}} d\alpha d\beta,$$

as we derived in the lecture. The last equation shows that we can obtain the posterior predictive distribution by averaging the predictive likelihood wrt. to the posterior distribution. Hence, our first effort will be to get a handle on the posterior distribution of α and β conditioned on $\mathbf{y}$.

For reference, the posterior over parameters is

$$p(\alpha, \beta \mid \mathbf{y}, \mathbf{x}) \propto p(\mathbf{y} \mid \mathbf{x}, \alpha, \beta) p(\alpha, \beta) = \left[\prod_{i=1}^M \text{Bin}(y_i \mid N, \theta_i) \right] \mathcal{N}(\alpha \mid 0, \sigma_\alpha^2) \mathcal{N}(\beta \mid 0, \sigma_\beta^2),$$

with proportionality constant $p(\mathbf{y} \mid \mathbf{x})^{-1}$.

(In logistic regression, this posterior has no closed form, so we typically approximate it via Laplace approximation, variational inference, or MCMC. The predictive integral can then be evaluated by Monte Carlo averaging.)

Quick notation reminders (to reduce cognitive load while reading equations):

- $N = N^* = 6$ throughout (six O-rings per case).
- $\theta_i = \sigma(\alpha + \beta x_i)$, and $\theta^* = \sigma(\alpha + \beta x^*)$.
- $f(x) = \alpha + \beta x$ is the linear predictor.
- $\theta(x) = \sigma(f(x))$.

- Bold symbols $\mathbf{x}, \mathbf{y}$ denote vectors over the M observed cases.

Before starting the practical implementation, we will practice the terminology and the probabilistic machinery.

Task 1.1: Use the **product rule** to write down the joint distribution of $\mathbf{y}, \alpha$, and β given $\mathbf{x}$:

Task 1.1 use a product rule to write down the joint distribution of $\mathbf{y}, \alpha, \beta$ given $\mathbf{x}$

Product Rule: $P(A, B) = P(A|B) P(B)$

- we can apply this to our case iteratively,

$$P(\mathbf{y}, \alpha, \beta | \mathbf{x}) = P(\mathbf{y} | \alpha, \beta, \mathbf{x}) \cdot P(\alpha, \beta | \mathbf{x})$$

- we can also go further and break up $P(\alpha, \beta | \mathbf{x})$:

$$P(\mathbf{y}, \alpha, \beta | \mathbf{x}) = P(\mathbf{y} | \alpha, \beta, \mathbf{x}) \cdot P(\alpha, \beta | \mathbf{x}) \cdot P(\beta | \mathbf{x})$$

- but α, β are $\perp$ of $\mathbf{x}$, before seeing the data.

$$\therefore \underbrace{P(\mathbf{y}, \alpha, \beta | \mathbf{x})}_{\text{Joint}} = \underbrace{P(\mathbf{y} | \alpha, \beta, \mathbf{x})}_{\text{Likelihood}} \cdot \underbrace{P(\alpha) \cdot P(\beta)}_{\text{priors}}$$

- Given $\mathbf{x}$ and some parameters α, β , how probable is the outcome $\mathbf{y}$?

Before seeing any data, how likely do we think various α, β values are?

We are modeling the prob of a binary outcome y
Example and our logistic regression model is:

$$\text{logit}(\theta_i) = \alpha + \beta_i$$

Our dataset is $\mathbf{x} = [1.2, 3.4, 0.7]$ and $\mathbf{y} = [0, 1, 0]$

- $P(\mathbf{y} | \alpha, \beta, \mathbf{x})$ = likelihood
- $P(\alpha) \cdot P(\beta)$ = beliefs about slope β and intercept α before data

Task 1.2: Show that $p(y, \alpha, \beta | x) = \sum_{y^*} p(y, y^*, \alpha, \beta | x, x^*)$, where the sum is over all possible outcomes of y^* .

Task 1.2 - Show that $p(y, \alpha, \beta | x) = \sum_{y^*} p(y, y^*, \alpha, \beta | x, x^*)$, where the sum is over all possible outcomes of y^* .

- we use the **Product Rule** to expand the **Joint**

$$p(y, y^*, \alpha, \beta | x, x^*) = p(y^* | x^*, \alpha, \beta) \cdot p(y | x, \alpha, \beta) p(\alpha, \beta)$$

- given α, β we assume y^*, y are conditionally $\perp$, as they are generated by the same model

- now we factor out terms independent of y^*

$$= p(y | x, \alpha, \beta) \cdot p(\alpha, \beta) \cdot \sum_{y^*} p(y^* | x^*, \alpha, \beta) \quad \sim \text{uses linearity of the summation operator}$$

- PMFs sum to 1

$$\sum_{y^*} p(y^* | x^*, \alpha, \beta) = 1 \quad \text{because } p(y^* | x^*, \alpha, \beta) \text{ is a valid PMF (Bernoulli)}$$

$$\therefore = p(y | x, \alpha, \beta) \cdot p(\alpha, \beta) \cdot 1$$

- now by a product rule:

$$p(y, \alpha, \beta | x) = p(y | x, \alpha, \beta) \cdot p(\alpha, \beta)$$

Task 1.3: Use Bayes' rule to express the posterior $p(\alpha, \beta | y, x)$ in terms of the prior, likelihood and evidence and argue that the posterior is proportional to the joint distribution $p(y, \alpha, \beta | x)$.

Task 1.3 Use Bayes rule to express the posterior $p(\alpha, \beta | y, x)$ in terms of the prior, likelihood and evidence and argue that the posterior is $\propto$ to the joint $p(y, \alpha, \beta, x)$

- use product rule:

$$p(y, \alpha, \beta | x) = p(y | \alpha, \beta, x) \cdot p(\alpha, \beta) \Rightarrow \text{we plug this into the Bayes Rule.}$$

$$p(\alpha, \beta | y, x) = \frac{p(y, \alpha, \beta | x)}{p(y | x)} = \frac{p(y | \alpha, \beta, x) \cdot p(\alpha, \beta)}{p(y | x)}$$

- because the evidence $p(y | x)$ is $\perp$ of the parameters, the post. is prop to joint lik.

$$p(\alpha, \beta | y, x) \propto p(y, \alpha, \beta | x) = p(y | \alpha, \beta, x) \cdot p(\alpha, \beta)$$

Part 2: The prior, likelihood and posterior

The goal of this part is to implement code for evaluating the prior, likelihood and posterior distribution. Due to a combination of **numerical precision** and **mathematical convenience**, we will implement functions for evaluating the logarithm of these distributions. Below you are given an incomplete implementation of the logistic regression model. Your task is to complete the implementation. The code contains a few sanity checks that will help to detect potential bugs and errors in your code.

Task 2.1: Complete the implementation of the functions `f`, `theta`, `log_prior` and `log_likelihood` below.

Hints:

- For implementing the log prior, the function `log_npdf` is handy.
- When implementing the log likelihood, you can either use `*scipy.stats.binom` (imported as `binom_dist`) or you can implement the PMF of the binomial distribution from scratch.*
- Note that for each function the output must have the same shape as the inputs. That is, if you run `log_prior(1, 1)`, it should return a scalar. If you run `log_prior(alpha_vec, beta_vec)`, where `alpha_vec` and `beta_vec` are $[10 \times 1]$ arrays, then the results should also be a $[10 \times 1]$ array

```
In [11]: import jax.numpy as jnp
from jax.scipy.stats import binom # Note: you'll need this import for binom_dist.logpmf

# =====
# UTILITY FUNCTIONS
# =====
```

```

# Sigmoid activation function:  $\sigma(x) = 1/(1 + \exp(-x))$ 
# Maps any real number to  $(0,1)$ , used to convert linear outputs to probabilities
# Input shape: (...) -> Output shape: (...)
sigmoid = lambda x: 1./(1 + jnp.exp(-x))

# Log probability density function for normal distribution
#  $\log N(x; \mu, \sigma^2) = -(x-\mu)^2/(2\sigma^2) - 0.5\log(2\pi\sigma^2)$ 
# Input shapes: x(...), m(...), v(...) -> Output shape: (...)
log_npdf = lambda x, m, v: -(x-m)**2/(2*v) - 0.5*jnp.log(2*jnp.pi*v)

# =====
# BAYESIAN LOGISTIC REGRESSION MODEL
# =====

# =====
# USAGE EXAMPLE
# =====

# Example data setup:
# x = jnp.array([1.0, 2.0, 3.0, 4.0]) # predictor variables
# y = jnp.array([1, 3, 7, 9]) # observed successes
# N = jnp.array([10, 10, 10, 10]) # trials per observation

# Instantiate model with default priors
model = LogisticRegression(x, y, N)

# The model can now be used for:
# 1. MAP estimation: optimize model.log_joint(alpha, beta)
# 2. MCMC sampling: use log_joint as target density
# 3. Variational inference: approximate posterior over  $(\alpha, \beta)$ 
# 4. Prediction: model.theta(x_new, alpha_sample, beta_sample)
# instantiate model
model = LR.LogisticRegression(x, y, N)

# some sanity checks to help verify your implementation and make it compatible with the rest of the exercise
assert jnp.allclose(model.theta(x=2, alpha=2, beta=-0.5), 0.7310585786300049), "The value of output of the theta-function was different than expected. Please check your code."
assert jnp.allclose(model.log_prior(1., 1.), -2.8378770664093453), "The value of the output of the log_prior function was different than expected. Please check your code."
assert jnp.allclose(model.log_likelihood(-1., 2.), -95.18926297085957), "The value of the output of the log_likelihood function was different than expected. Please check your code."

assert model.theta(jnp.linspace(-3, 3, 10), 1, 1).shape == (10,), "The shape of the output of the theta-function was different than expected. Please check your code."
assert model.log_prior(jnp.linspace(-3, 3, 10), jnp.linspace(-3, 3, 10)).shape == (10,), "The shape of the output of the log_prior-function was different than expected. Please check your code."
assert model.log_likelihood(jnp.linspace(-3, 3, 10)[, None], jnp.linspace(-3, 3, 10)[, None]).shape == (10, 1), "The shape of the output of the log_likelihood-function was different than expected. Please check your code."

```

Use Case: Bayesian Logistic Regression with Binomial Data

Let's suppose you're modeling the number of successes (e.g., passes in an exam) out of a fixed number of trials for different students based on their study time. You suspect that the probability of success increases with study time and want to model this probabilistically, incorporating uncertainty over the parameters.

```

In [6]: # prepare the data

x = jnp.array([1.0, 2.0, 3.0, 4.0]) # hours studied (input features)
y = jnp.array([1, 3, 7, 9]) # number of passes (successes)
N = jnp.array([10, 10, 10, 10]) # number of exams (trials)

# instantiate the model
model_use_case = LR.LogisticRegression(x=x, y=y, N=N, sigma2_alpha=1.0, sigma2_beta=1.0)

```

Step 3: Compute Predictions with θ

► What does `theta()` do? It computes the predicted probability of success for a given input using:

$$\theta(x) = \sigma(\alpha + \beta x),$$

$$\text{where } \sigma(z) = \frac{1}{1+e^{-z}}$$

It can be used to **visualize the decision boundary** or **make predictions** with sampled parameters.

```

In [7]: # example: predict the success probability for x = 2, with alpha = 2, beta = -0.5
prob_use_case = model_use_case.theta(2, alpha=2, beta=-0.5)
print(prob_use_case) # ≈ 0.731 (sigmoid(2 - 1))

0.73105854

```

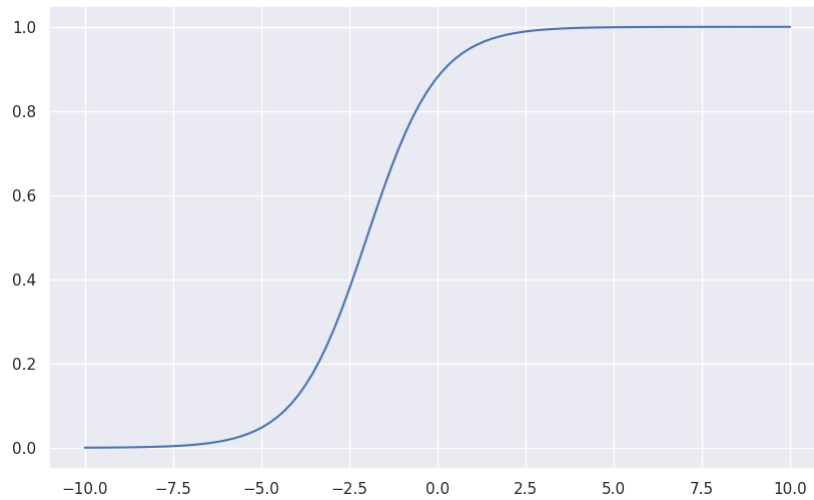
```

In [9]: # You can also use this in batch mode:
x_grid_use_case = jnp.linspace(-10, 10, 100)
probs_use_case_2 = model.theta(x_grid_use_case, alpha=2.0, beta=1.0)

```

```
fig, ax = plt.subplots(1, figsize=(10, 6))
ax.plot(x_grid_use_case, probs_use_case_2)
```

Out[9]: [<matplotlib.lines.Line2D at 0x7f4c487bde50>]



Step 4: Compute Log Prior with `log_prior` ► What does `log_prior()` do?

It computes the log of the prior probability:

$$\log p(\alpha, \beta) = \log \mathcal{N}(\alpha, 0, \sigma_\alpha^2) + \log \mathcal{N}(\beta, 0, \sigma_\beta^2)$$

Use this when you want to penalize large parameter values, or to compute the posterior later via Bayes' rule.

```
In [10]: # Example: log prior probability of a = 1, beta = 1
loggp = model.log_prior(1., 1.)
print(loggp) # ≈ -2.83
-2.837877
```

In []:

```
In [11]: # You can also pass arrays to compute this for many values:
alphas = jnp.linspace(-3, 3, 10)
betas = jnp.linspace(-3, 3, 10)
log_priors = model.log_prior(alphas, betas) # shape: (10,)
print(log_priors)
print(f"Shape of the log prior {log_priors.shape}")
[-10.837877 -7.282322 -4.615655 -2.8378768 -1.9489882 -1.9489882
 -2.837877 -4.6156554 -7.282322 -10.837877 ]
Shape of the log prior (10,)
```

Step 5: Compute Log Likelihood with `log_likelihood` ► What does `log_likelihood()` do?

This computes the log probability of the observed data under the model, assuming:

$$y_i \mid \text{Binomial}(N_i, \theta_i) \quad \text{where} \quad \theta_i = \theta(x_i) = \sigma(\alpha + \beta x_i) = \frac{1}{1 + \exp(-(\alpha + \beta x_i))}.$$

This is our likelihood function, which we will use to compute the posterior distribution.

$$\log p(y \mid \alpha, \beta) = \sum_{i=1}^N \log \text{Binomial}(y_i \mid N_i, \theta_i) = \sum_{i=1}^N \left(\log \binom{N_i}{y_i} + y_i \log \theta_i + (N_i - y_i) \log(1 - \theta_i) \right)$$

```
In [12]: log_lik_use_case = model_use_case.log_likelihood(alpha=-1., beta=2.)
print(log_lik_use_case)
[-41.51465]
```

```
In [13]: # For grid evaluation:

alphas_use_case = jnp.linspace(-3, 3, 10)[:, None]
betas_use_case = jnp.linspace(-3, 3, 10)[:, None]
log_lik_use_case = model.log_likelihood(alphas_use_case, betas_use_case) # shape: (10, 1)
print(log_lik_use_case)
print(f"Log likelihood use case shape = {log_lik_use_case.shape}")
```

```
[[ -53.163406]
 [ -45.3126  ]
 [ -41.038307]
 [ -44.073666]
 [ -62.532593]
 [-107.563705]
 [-179.16695 ]
 [-266.19385 ]
 [-360.53027 ]
 [-458.44345 ]]
Log likelihood use case shape = (10, 1)
```

 **Step 6: Compute Log Joint Probability with log_joint** ➤ What does log_joint() do?

It computes the unnormalized log posterior:

$$\log p(\alpha, \beta | y) \propto \log p(y | \alpha, \beta) + \log p(\alpha, \beta)$$

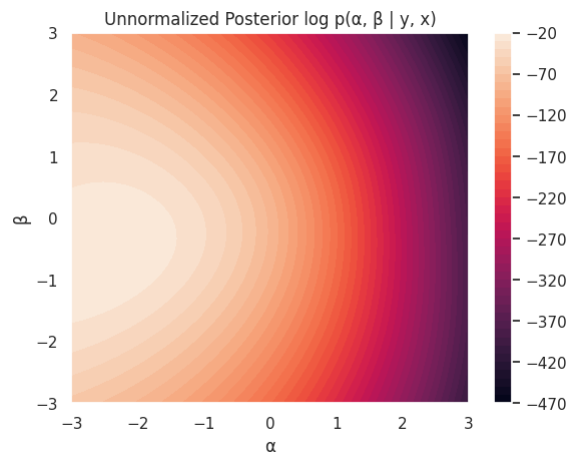
```
In [14]: log_joint = model.log_joint(alpha=1., beta=1.)
print(log_joint) # A scalar value (e.g., -97.6)

-182.00482
```

```
In [15]: # this can be also computed over a grid for contour plots
import matplotlib.pyplot as plt
alpha_grid = jnp.linspace(-3, 3, 100)
beta_grid = jnp.linspace(-3, 3, 100)
AA, BB = jnp.meshgrid(alpha_grid, beta_grid)

log_post = jnp.vectorize(lambda a, b: model.log_joint(a, b))(AA, BB)

plt.contourf(AA, BB, log_post, levels=50)
plt.xlabel("α")
plt.ylabel("β")
plt.title("Unnormalized Posterior log p(α, β | y, x)")
plt.colorbar()
plt.show()
```



If the model implementation above is correct, then the code below should produce contour plots of the prior, likelihood and posterior. We also compute and plot the MLE and MAP estimators for the model. Because this is a 2D problem, we can simply obtain the MLE/MAP estimators by maximizing over the set of grid values. However, this becomes infeasible in higher dimensions, where gradient-based optimization is used instead.

```
In [ ]:
```

 **What Grid2D Does**

In Bayesian logistic regression, we're interested in understanding how the posterior distribution over parameters α and β behaves.

The `Grid2D` class is a helper to evaluate and visualize any function of (α, β) on a 2D grid:

-  Prior: $p(\alpha, \beta)$
-  Likelihood: $p(y \mid \alpha, \beta)$
-  Posterior: $p(\alpha, \beta \mid y) \propto p(y \mid \alpha, \beta) \cdot p(\alpha, \beta)$

What It Computes

```
self.values = self.func(self.alpha_grid[:, :, None], self.beta_grid[:, :, None]).squeeze()
```

Each element in `self.values[i, j]` stores the value of

$f(\alpha_i, \beta_j)$

Where:

- f is one of: `log_prior`, `log_likelihood`, or `log_joint` (posterior)
- `self.alpha_grid[i, j] =  $\alpha_j$` , `self.beta_grid[i, j] =  $\beta_i$`

Full Working Example with Explanations

Step 1: Prepare the model

```
In [16]: # Inputs
x_g = jnp.array([1.0, 2.0, 3.0, 4.0]) # Input features
y_g = jnp.array([1, 3, 7, 9])         # Observed successes
N_g = jnp.array([10, 10, 10, 10])     # Trials

# Instantiate your Bayesian logistic regression model
model_g = LR.LogisticRegression(x_g, y_g, N_g, sigma2_alpha=1.0, sigma2_beta=1.0)
```

Step 2: Create the 2D parameter grid

```
In [17]: num_alpha_g, num_beta_g = 90, 100
alphas_g = jnp.linspace(-4, 4, num_alpha_g) #  $\alpha$  axis
betas_g = jnp.linspace(-4, 4, num_beta_g)   #  $\beta$  axis
```

Step 3: Evaluate Prior, Likelihood, and Posterior

```
In [18]: # Prior: Encodes belief before seeing data
log_prior_grid = LR.Grid2D(alphas=alphas_g, betas=betas_g, func=model.log_prior, name="Prior")

# Likelihood: How well parameters explain observed data
log_lik_grid = LR.Grid2D(alphas=alphas_g, betas=betas_g, func=model.log_likelihood, name="Likelihood")

# Posterior: Combines data + prior belief
log_joint_grid = LR.Grid2D(alphas=alphas_g, betas=betas_g, func=model.log_joint, name="Posterior")
```

Step 4: Extract MAP and MLE Estimates

```
In [19]: # Maximum Likelihood Estimate (ignores prior)
alpha_MLE_g, beta_MLE_g = log_lik_grid.argmax

# Maximum a Priori (include prior)
alpha_MAP_g, beta_MAP_g = log_joint_grid.argmax

print(f'MLE:  $\alpha$  = {alpha_MLE_g:.3f},  $\beta$  = {beta_MLE_g:.3f}')
print(f'MAP:  $\alpha$  = {alpha_MAP_g:.3f},  $\beta$  = {beta_MAP_g:.3f}')

MLE:  $\alpha$  = -2.921,  $\beta$  = -0.768
MAP:  $\alpha$  = -2.562,  $\beta$  = -0.606
```

Step 5: Visualize the Grids

```
In [20]: import matplotlib.pyplot as plt
import jax.numpy as jnp # Assuming you're using JAX

# Colors for each component
colors_g = ['blue', 'red', 'green']

# Create figure and axes
fig_g, ax_g = plt.subplots(1, 3, figsize=(20, 6))

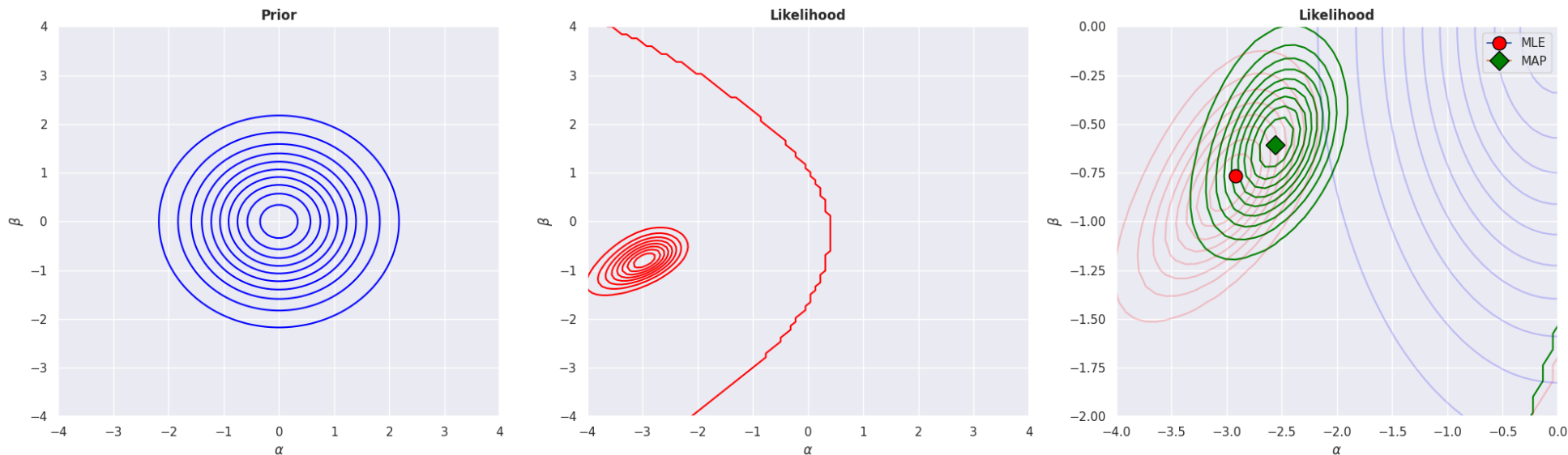
# Plot each component
for i, grid in enumerate([log_prior_grid, log_lik_grid, log_joint_grid]):
    grid.plot_contours(ax_g[i], f=jnp.exp, color=colors_g[i])
```

```
# Overlay prior and likelihood on posterior for intuition
log_prior_grid.plot_contours(ax_g[2], f=jnp.exp, color='blue', alpha=0.2)
log_lik_grid.plot_contours(ax_g[2], f=jnp.exp, color='red', alpha=0.2)

# Mark MLE and MAP
ax_g[2].plot(alpha_MLE_g, beta_MLE_g, marker='o', markersize=12, markerfacecolor='red', markeredgecolor='black', label='MLE')
ax_g[2].plot(alpha_MAP_g, beta_MAP_g, marker='D', markersize=12, markerfacecolor='green', markeredgecolor='black', label='MAP')

# Set limits and legend
ax_g[2].set_xlim(-4, 0)
ax_g[2].set_ylim(-2, 0)
ax_g[2].legend()

plt.tight_layout()
plt.show()
```



In []:

In []:

```
In [12]: # =====
# GRID SETUP AND FUNCTION EVALUATION
# =====

# Define grid points for parameter space exploration
# Different numbers of points for alpha and beta makes debugging easier
num_alpha, num_beta = 90, 100
alphas = jnp.linspace(-4, 4, num_alpha) # Shape: (90,) - intercept parameter range
betas = jnp.linspace(-4, 4, num_beta)    # Shape: (100,) - slope parameter range

# Create Grid2D objects for each component of Bayesian analysis
# Assumes 'model' is a LogisticRegression object with methods:
# - log_prior(alpha, beta): log P(alpha, beta)
# - log_likelihood(alpha, beta): log P(data | alpha, beta)
# - log_joint(alpha, beta): log P(alpha, beta | data) = log_prior + log_likelihood

model = LR.LogisticRegression(x, y, N, sigma2_alpha=1.0, sigma2_beta=1.0)

# Evaluate log prior distribution: log P(alpha, beta)
# Typically: log P(alpha) + log P(beta) where P(alpha) ~ N(0, sigma2_alpha), P(beta) ~ N(0, sigma2_beta)
log_prior_grid = LR.Grid2D(alphas, betas, model.log_prior, 'Prior')

# Evaluate log likelihood: log P(data | alpha, beta) = \sum log P(y_i | x_i, alpha, beta)
# For logistic regression: \sum [y_i * log(pi) + (1-y_i) * log(1-pi)] where pi = sigmoid(alpha + beta * x_i)
log_lik_grid = LR.Grid2D(alphas, betas, model.log_likelihood, 'Likelihood')

# Evaluate log posterior (joint): log P(alpha, beta | data) \propto log P(data | alpha, beta) + log P(alpha, beta)
```

```

log_joint_grid = LR.Grid2D(alphas, betas, model.log_joint, 'Posterior')

# =====
# VISUALIZATION
# =====

# Create subplot layout: [Prior, Likelihood, Posterior]
fig, ax = plt.subplots(1, 3, figsize=(20, 6))

# Plot prior and likelihood contours on the posterior plot (rightmost) for comparison
# f=jnp.exp converts from log-space to probability space for better visualization
# alpha=0.2 makes these overlay contours semi-transparent
log_prior_grid.plot_contours(ax[2], f=jnp.exp, color='b', alpha=0.2)      # Blue, transparent
log_lik_grid.plot_contours(ax[2], f=jnp.exp, color='r', alpha=0.2)        # Red, transparent

# Plot each distribution on its own subplot
# colors[idx_plot] should be defined elsewhere (e.g., ['blue', 'red', 'green'])
for idx_plot, grid in enumerate([log_prior_grid, log_lik_grid, log_joint_grid]):
    grid.plot_contours(ax[idx_plot], f=jnp.exp, color=colors[idx_plot])

# =====
# POINT ESTIMATES
# =====

# Compute Maximum Likelihood Estimate (MLE): argmax P(data | alpha, beta)
# This finds the parameters that maximize the likelihood of observed data
alpha_MLE, beta_MLE = log_lik_grid.argmax

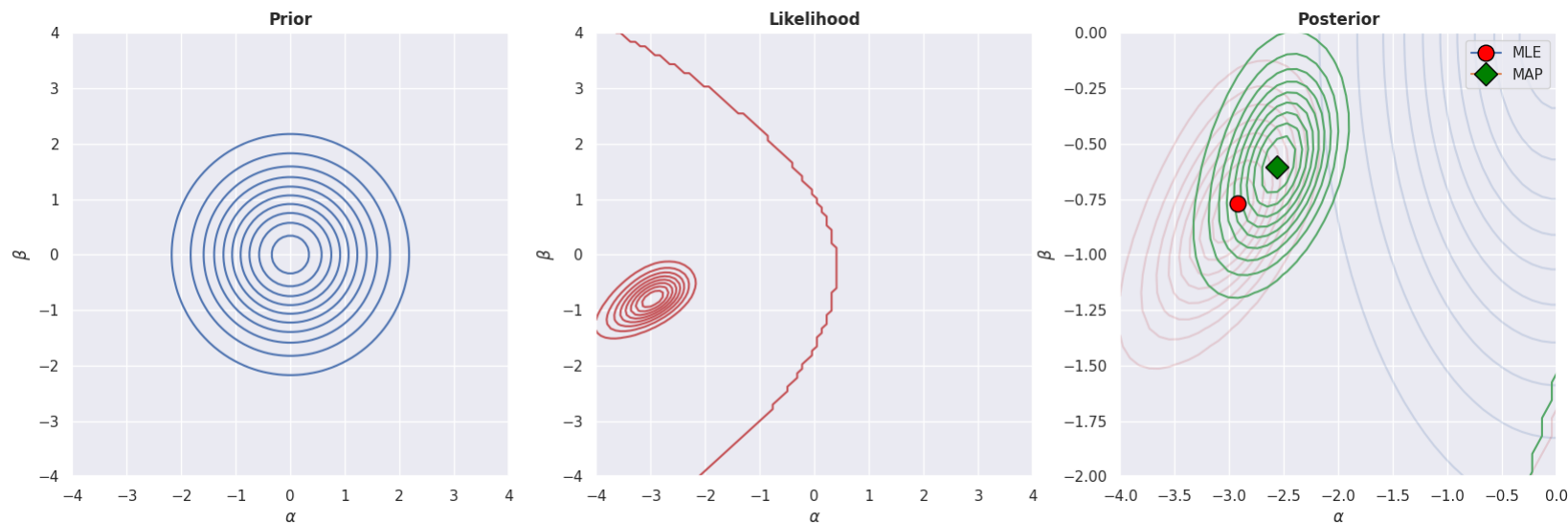
# Compute Maximum A Posteriori (MAP) estimate: argmax P(alpha, beta | data)
# This finds the parameters that maximize the posterior distribution
# MAP incorporates both data likelihood and prior beliefs
alpha_MAP, beta_MAP = log_joint_grid.argmax

# Plot point estimates on the posterior subplot
# Circle with filled color and edge
ax[2].plot(alpha_MLE, beta_MLE, marker='o', markersize=12,
            markerfacecolor='red', markeredgecolor='black', label='MLE')

ax[2].plot(alpha_MAP, beta_MAP, marker='D', markersize=12,
            markerfacecolor='green', markeredgecolor='black', label='MAP')
ax[2].set_xlim(-4, 0)
ax[2].set_ylim(-2, 0)
# add the plot label for prior
ax[2].legend()

plt.show()
# Key Insights:
# - MLE ignores prior information, purely data-driven
# - MAP balances data likelihood with prior beliefs
# - If prior is uninformative, MLE = MAP
# - If data is limited, MAP will be closer to prior mean than MLE

```



Task 2.2: When plotting the contours of the posterior distribution, we actually evaluate the joint distribution and not the posterior distribution. Why is this okay when making contour plots?

Bayes' rule states that the posterior is proportional to the joint distribution. That is, $p(\alpha, \beta | \mathbf{y}, \mathbf{x}) = \frac{p(\mathbf{y} | \alpha, \beta, \mathbf{x}) p(\alpha, \beta)}{p(\mathbf{y} | \mathbf{x})} \propto p(\mathbf{y} | \alpha, \beta, \mathbf{x}) p(\alpha, \beta)$ because the evidence $p(\mathbf{y} | \mathbf{x})$ is constant wrt. α and β and hence does not affect the contour plots.

Task 2.3: Use the code above to experiment and investigate the relationship between the prior, likelihood and posterior. What happens if you change the prior variances of α, β ? E.g. $\sigma_\alpha^2 = \sigma_\beta^2 = 10$, $\sigma_\alpha^2 = \sigma_\beta^2 = 100$, $\sigma_\alpha^2 = \sigma_\beta^2 = 0.1$? What happens if $\sigma_\alpha^2 = 1$ and $\sigma_\beta^2 = 10$?

Base Formulas Recap

Let's work in log-space since that's what's used in the code and grid:

◆ **1. Log-likelihood** (unchanged — depends only on data):

$$\log p(\mathbf{y} | \mathbf{x}, \alpha, \beta) = \sum_{i=1}^N [y_i \log \sigma(\alpha + \beta x_i) + (1 - y_i) \log(1 - \sigma(\alpha + \beta x_i))]$$

◆ **2. Log-prior** (this changes with σ_α^2 and σ_β^2):

Assume: $\alpha \sim \mathcal{N}(0, \sigma_\alpha^2)$, $\beta \sim \mathcal{N}(0, \sigma_\beta^2)$

Then:

$$\log p(\alpha, \beta) = -\frac{1}{2} \left(\frac{\alpha^2}{\sigma_\alpha^2} + \frac{\beta^2}{\sigma_\beta^2} \right) + \text{const}$$

So **changing** σ_α^2 or σ_β^2 changes the penalty term:

- If σ^2 is **small**, the penalty is **large**, which **pulls** α or β **closer to zero**.
- If σ^2 is **large**, the penalty is **small**, so the prior becomes **flatter**.

● **3. Log-posterior (this is the key):**

$$\log p(\alpha, \beta | \mathbf{x}, \mathbf{y}) = \log p(\mathbf{y} | \mathbf{x}, \alpha, \beta) + \log p(\alpha, \beta) + \text{const}$$

If we substitute in the equations:

$$\log p(\alpha, \beta | \mathbf{x}, \mathbf{y}) = \sum_{i=1}^N [y_i \log \sigma(\alpha + \beta x_i) + (1 - y_i) \log(1 - \sigma(\alpha + \beta x_i))] - \frac{1}{2} \left(\frac{\alpha^2}{\sigma_\alpha^2} + \frac{\beta^2}{\sigma_\beta^2} \right) + \text{const}$$

This is the **exact formula** used in your code's `log_joint`.

Effect of Changing Prior Variances

Let's now see how the equation changes for each case:

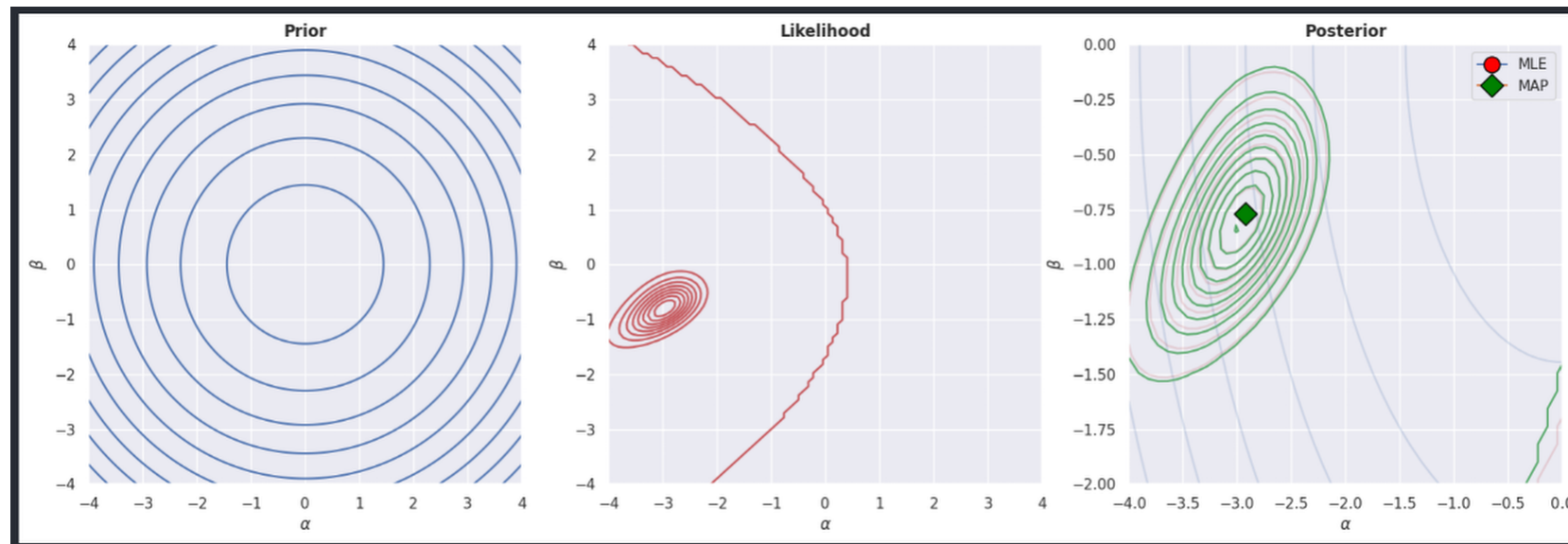
✓ **Case 1:** $\sigma_\alpha^2 = \sigma_\beta^2 = 100$

$$\log p(\alpha, \beta) = -\frac{1}{2} \left(\frac{\alpha^2}{100} + \frac{\beta^2}{100} \right)$$

👉 The penalty term becomes:

$$-\frac{1}{200}(\alpha^2 + \beta^2)$$

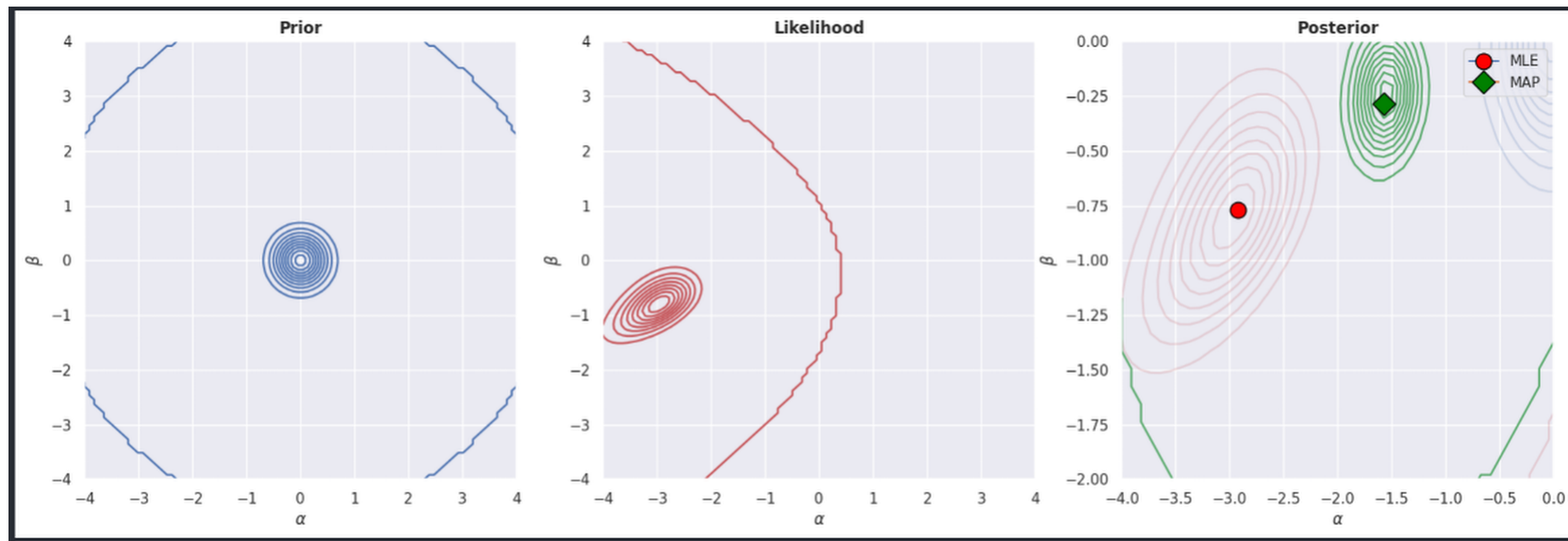
- This is a **very small penalty**, so the prior is almost flat
- The log-posterior $\approx$ log-likelihood
- MAP $\approx$ MLE



✓ **Case 2:** $\sigma_\alpha^2 = \sigma_\beta^2 = 0.1$

$$\log p(\alpha, \beta) = -\frac{1}{2} \left(\frac{\alpha^2}{0.1} + \frac{\beta^2}{0.1} \right) = -5(\alpha^2 + \beta^2)$$

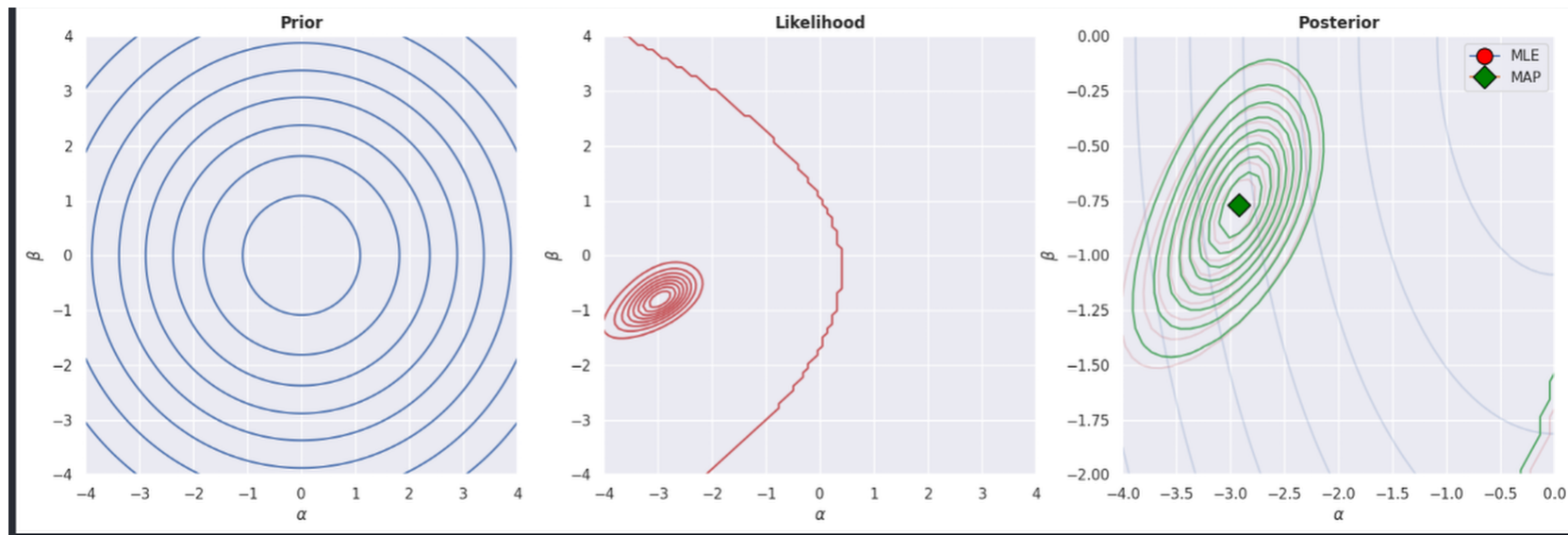
- This is a **strong penalty**, heavily pulling (α, β) toward 0
- The posterior is **dominated by the prior**
- MAP is **far from MLE**, closer to prior mean (which is 0)



✓ Case 3: $\sigma_\alpha^2 = 1, \sigma_\beta^2 = 10$

$$\log p(\alpha, \beta) = -\frac{1}{2} \left(\alpha^2 + \frac{\beta^2}{10} \right)$$

- The prior penalizes α more than β
- So the MAP estimate is more conservative (closer to 0) in α
- The posterior is more squeezed along the α -axis



✦ Summary Table of Log-Prior Term:

σ^2_{α}	σ^2_{β}	Penalty term in log-prior	Interpretation
100	100	$-\frac{1}{200}(\alpha^2 + \beta^2)$	Almost flat prior
0.1	0.1	$-5(\alpha^2 + \beta^2)$	Strong prior pulling to 0
1	10	$-\frac{1}{2}(\alpha^2 + \frac{\beta^2}{10})$	Prior stronger on α

Task 2.4: What happens to the MLE and MAP estimators when you change the variances of the prior?

✔ Task 2.4 Answer

🧠 What happens to the MLE and MAP estimators when you change the prior variances?

🔴 MLE is unchanged by the prior variance.

• Definition:

$$(\alpha_{\text{MLE}}, \beta_{\text{MLE}}) = \arg \max_{\alpha, \beta} \log p(\mathbf{y} \mid \mathbf{x}, \alpha, \beta)$$

- Only depends on the data via the likelihood.
- So no matter what prior you choose, MLE will stay the same.

🟢 MAP is affected by the prior variance.

• Definition:

$$(\alpha_{\text{MAP}}, \beta_{\text{MAP}}) = \arg \max_{\alpha, \beta} [\log p(\mathbf{y} \mid \mathbf{x}, \alpha, \beta) + \log p(\alpha, \beta)]$$

- The second term,

$$\log p(\alpha, \beta) = -\frac{1}{2} \left(\frac{\alpha^2}{\sigma_{\alpha}^2} + \frac{\beta^2}{\sigma_{\beta}^2} \right) + \text{const}$$

acts as a **regularizer** that discourages large values of α and β .

📄 Summary of Effects

Prior variance	Effect on log prior	Effect on MAP estimate	MLE
$\sigma^2 \rightarrow \infty$	Prior becomes flat (uninformative)	MAP $\approx$ MLE	Unchanged
σ^2 small	Prior becomes sharp around 0	MAP is pulled toward 0	Unchanged
Asymmetric (e.g. $\sigma_{\alpha}^2 = 1, \sigma_{\beta}^2 = 10$)	Penalizes α more than β	MAP is closer to 0 in α	Unchanged

📖 Connection to the Equation (again, explicitly)

Posterior log-probability:

$$\log p(\alpha, \beta \mid \mathbf{x}, \mathbf{y}) = \underbrace{\log p(\mathbf{y} \mid \mathbf{x}, \alpha, \beta)}_{\text{Likelihood (data)}} + \underbrace{\log p(\alpha, \beta)}_{\text{Prior (regularizer)}}$$

Substituting the Gaussian prior:

$$\log p(\alpha, \beta \mid \mathbf{x}, \mathbf{y}) = \sum_{i=1}^N [y_i \log \sigma(\alpha + \beta x_i) + (1 - y_i) \log (1 - \sigma(\alpha + \beta x_i))] - \frac{1}{2} \left(\frac{\alpha^2}{\sigma_{\alpha}^2} + \frac{\beta^2}{\sigma_{\beta}^2} \right) + \text{const}$$

- The **MLE ignores the prior**, so it just maximizes the first sum.
- The **MAP includes the prior penalty**, so:
 - Small variances **increase** the penalty → MAP is more regularized.
 - Large variances **reduce** the penalty → MAP moves **closer to MLE**.

Intuition

- MAP is a **regularized** version of MLE.
- The **prior variance controls the strength of regularization**:
 - Smaller σ^2 = stronger regularization = more trust in prior.
 - Larger σ^2 = weaker regularization = more trust in data.

Task 2.5: What happens to the prior, likelihood and posterior if you only use a subset of the data? Use the code above to experiment and investigate what happens, e.g. use the first 10 data instead of the full $M = 23$ datapoints.

1. The Prior:

Unchanged.

It is **independent of the data**, defined as:

$$p(\alpha, \beta) = \mathcal{N}(0, \sigma_\alpha^2) \cdot \mathcal{N}(0, \sigma_\beta^2)$$

So in code:

```
model = LR.LogisticRegression(x_subset, y_subset, N, sigma2_alpha, sigma2_beta)
log_prior_grid = Grid2D(alphas, betas, model.log_prior, "Prior")
# The prior will be the same regardless of how many data points you use.
```

2. The Likelihood:

Log-likelihood is:

$$\log p(\mathbf{y} \mid \mathbf{x}, \alpha, \beta) = \sum_{i=1}^N [y_i \log \sigma(\alpha + \beta x_i) + (1 - y_i) \log(1 - \sigma(\alpha + \beta x_i))]$$

✂ Using fewer datapoints (e.g. first 10 instead of 23):

- This **reduces the sum**, so the likelihood becomes:
 - Less peaked** (less confident)
 - Flatter**, more spread out
- You're working with less evidence, so the model is more **uncertain** about which parameters are optimal.

3. The Posterior:

Posterior (in log-space) is:

$$\log p(\alpha, \beta \mid \mathbf{y}, \mathbf{x}) = \log \text{Likelihood} + \log \text{Prior}$$

- Since the **likelihood is flatter**, the **prior becomes more influential** in the posterior.
- So the **posterior shifts closer to the prior**.

That means:

- The **MAP estimate moves toward (0, 0)**.
- The **posterior is broader**, less peaked.

📊 Summary Table

Component	Mathematical Change	Code Effect	Intuition
Prior	None (fixed)	Unchanged	Still centered at 0
Likelihood	Shorter sum over data	Flatter, less informative	More uncertainty about best (α, β)
Posterior	Prior becomes dominant	Posterior peak shifts toward (0, 0)	MAP $\neq$ MLE becomes more pronounced

🧠 Conceptual Takeaway

With less data, the likelihood becomes weaker, and the prior plays a stronger role in shaping the posterior. That's the whole idea behind Bayesian reasoning: when data is scarce, trust your prior more.

Task 2.6: Use the plots to roughly estimate the prior and posterior probability of the event $\beta < 0$.

```
In [22]: fig, ax = plt.subplots(1, 1, figsize=(20, 6))
# plot the likelihood contours
```

```

log_lik_grid.plot_contours(ax, f=jnp.exp, color='r', alpha=0.2)
# Plot prior contours
log_prior_grid.plot_contours(ax, f=jnp.exp, color='b', alpha=0.2)

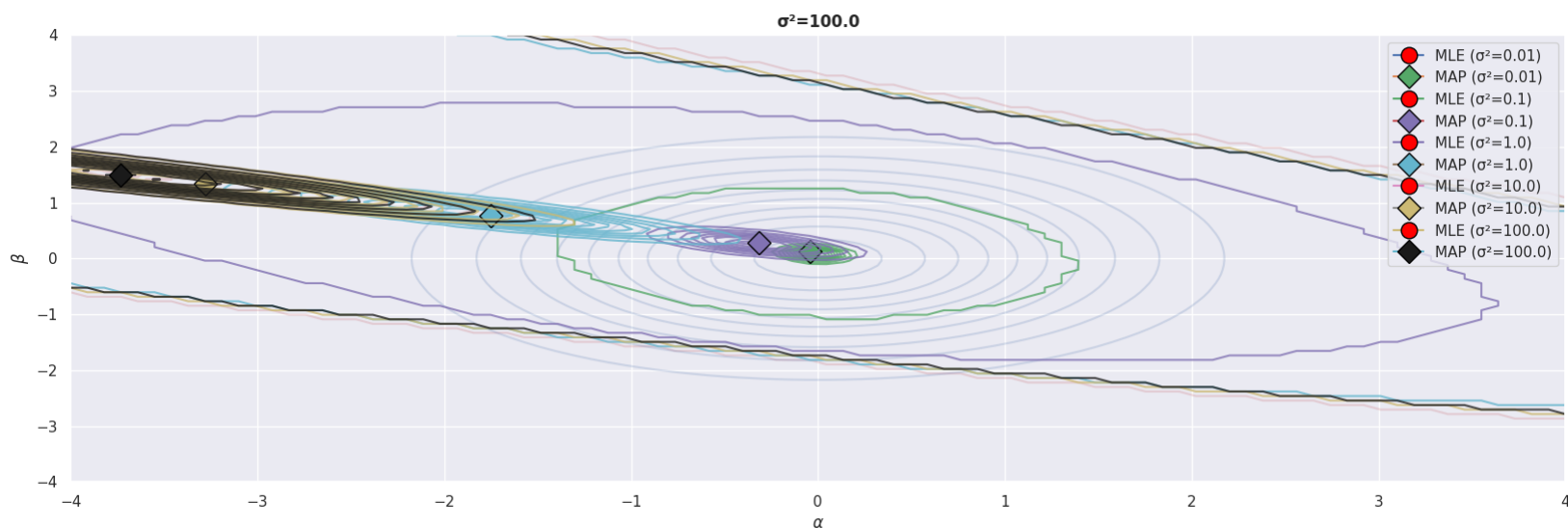
color_list = ['g', 'm', 'c', 'y', 'k']
for idx, s in enumerate([0.01, 0.1, 1.0, 10.0, 100.0]):
    model = LR.LogisticRegression(x, y, N, sigma2_alpha=s, sigma2_beta=s)
    log_joint = LR.Grid2D(alphas, betas, model.log_joint, f='σ²={s}')
    # Plot on the posterior subplot (ax)
    log_joint.plot_contours(ax, f=jnp.exp, color=color_list[idx], alpha=0.8, title=f'σ²={s}')

    # Plot point estimates for each σ²
    # MLE is always the same (from likelihood only)
    alpha_MLE, beta_MLE = log_lik_grid.argmax
    # MAP depends on the prior
    alpha_MAP, beta_MAP = log_joint.argmax

    # Plot MLE and MAP estimates with different colors and labels
    ax.plot(alpha_MLE, beta_MLE, marker='o', markersize=12,
            markerfacecolor='red', markeredgecolor='black',
            label=f'MLE (σ²={s})')
    ax.plot(alpha_MAP, beta_MAP, marker='D', markersize=12,
            markerfacecolor=color_list[idx], markeredgecolor='black',
            label=f'MAP (σ²={s})')

ax.legend()
plt.show()

```



Part 3: Making predictions using point estimates

Our model for the probability of failure for each individual o-ring is given by

$$\theta(x) = \sigma(\alpha + \beta x),$$

where $\alpha, \beta \in \mathbb{R}$ are unknown parameters. Since we do not know the values of the parameters, we can **estimate** them from the data in several ways. The simplest way is via the maximum likelihood estimator (MLE). The idea is to find the point (α, β) that makes the observed data most probable under the model. Formally, we define the likelihood of the data:

$$\hat{\alpha}_{\text{MLE}}, \hat{\beta}_{\text{MLE}} = \arg \max_{\alpha, \beta \in \mathbb{R}} p(\mathbf{y} | \mathbf{x}, \alpha, \beta).$$

In practice, we most often do this using gradient-based optimization. However, in this exercise, we will simply pick the parameters values that maximizes the likelihood function on the grid, which works well for 2D problems. Instead of maximizing the likelihood, we may instead want to incorporate prior beliefs about the parameters. This leads to the Maximum A Posteriori (MAP) estimator, where we maximize the posterior distribution:

$$\hat{\alpha}_{\text{MAP}}, \hat{\beta}_{\text{MAP}} = \arg \max_{\alpha, \beta} p(\alpha, \beta | \mathbf{y}, \mathbf{x}).$$

This estimator combines the likelihood with a prior using Bayes' rule:

$$p(\alpha, \beta | \mathbf{y}, \mathbf{x}) \propto p(\mathbf{y} | \mathbf{x}, \alpha, \beta) p(\alpha, \beta)$$

These point estimates $(\hat{\alpha}, \hat{\beta})$ can then be used to make deterministic predictions. For instance, for a new input x^* , we plug them into the model:

$$\hat{\theta}(x^*) \approx \sigma(\hat{\alpha} + \hat{\beta}x^*)$$

and use this estimated probability to form the predictive distribution:

$$p(y^* | x^*, \hat{\alpha}, \hat{\beta}) = \text{Bin}(y^* | N^*, \hat{\theta}(x^*))$$

 **Limitation:** While this is computationally convenient, it ignores uncertainty in the parameters. Later, we'll compare this with the full posterior predictive that integrates over (α, β) , which captures parameter uncertainty explicitly.

Task 3.1: Compute and plot the predicted probability $\hat{\theta}(x)$ for x corresponding to temperatures $t \in [-10, 40]$ for both MLE and MAP estimators.

```
In [13]: # data for predictions
temp_pred = jnp.linspace(-10, 40, 500)
x_pred = standardize(temp_pred)
N_pred = 6

MLE = log_lik_grid.argmax
print(MLE)

MAP = log_joint_grid.argmax
print(MAP)

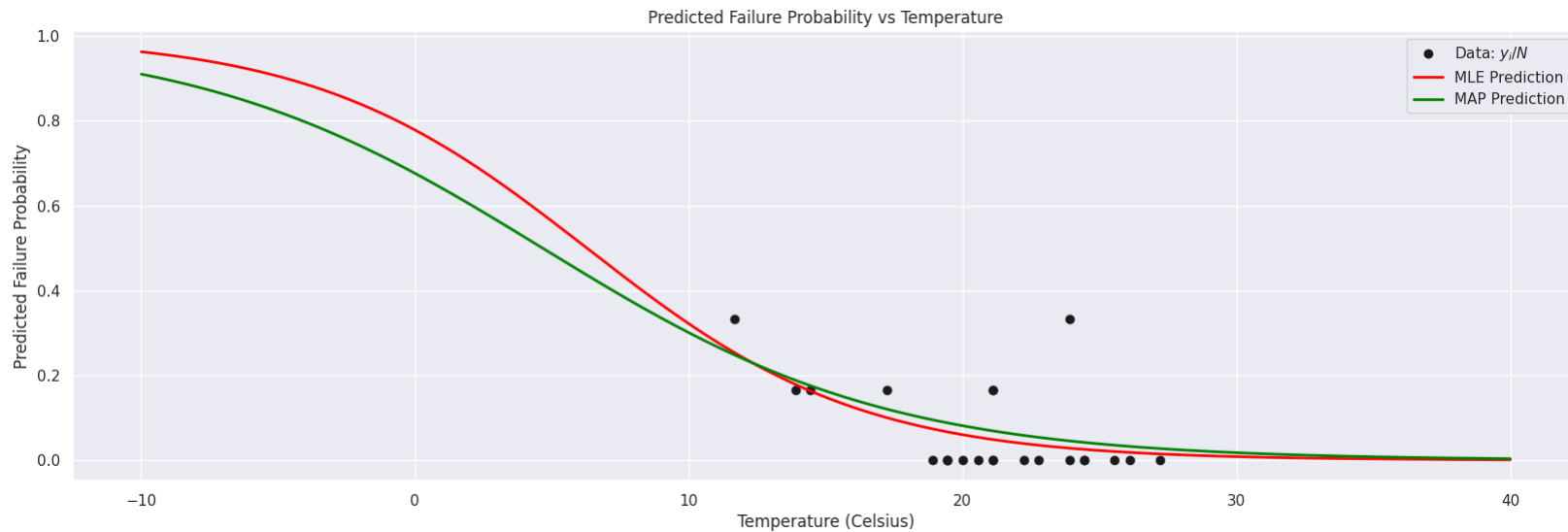
MLE_pred = sigmoid(MLE[0] + MLE[1] * x_pred)
MAP_pred = sigmoid(MAP[0] + MAP[1] * x_pred)

# plot the predictions

fig, ax = plt.subplots(1, 1, figsize=(20, 6))
# Plot the data
plot_data(ax, counts=False)

# Plot MLE predictions
ax.plot(temp_pred, MLE_pred, color='red', label='MLE Prediction', linewidth=2)
# Plot MAP predictions
ax.plot(temp_pred, MAP_pred, color='green', label='MAP Prediction', linewidth=2)
# Add labels and legend
ax.set_xlabel('Temperature (Celsius)')
ax.set_ylabel('Predicted Failure Probability')
ax.set_title('Predicted Failure Probability vs Temperature')
ax.legend()
plt.show()

(Array(-2.9213486, dtype=float32), Array(-0.76767683, dtype=float32))
(Array(-2.5617976, dtype=float32), Array(-0.6060605, dtype=float32))
```



Part 4: Approximating the posterior using a grid approximation

Rather than using point estimates (such as **MLE** or **MAP**), we now turn to **full Bayesian inference**, which allows us to reason about **uncertainty** in both parameters and predictions.

However, unlike the **Beta-binomial** model, logistic regression does **not** yield a conjugate posterior, meaning that the posterior distribution

$$p(\alpha, \beta \mid \mathbf{y}, \mathbf{x})$$

cannot be computed analytically. Instead, we approximate it using a **discrete grid-based representation** over the parameter space, denoted as

$$p(\alpha, \beta \mid \mathbf{y}, \mathbf{x}) \approx q(\alpha, \beta).$$

This grid approximation makes it possible to **summarize and work with the posterior distribution** by evaluating it over a finite set of (α, β) values.

To construct the grid approximation $q(\alpha, \beta)$, we need three steps:

1. **Define grid:** First we need to define the grid for (α, β) . Typically, we choose an appropriate discrete set of values for α and β individually, i.e. $\{\alpha_i\}_{i=1}^I$ and $\{\beta_j\}_{j=1}^J$, and then take the (α, β) -grid to be the Cartesian product. This gives us $I \times J$ evaluation points in total. The Python class `Grid2D` is responsible for generating this grid and evaluating a function (e.g. the log-joint probability) at each grid point.
2. **Evaluation:** Next, we need to evaluate the function $p(\alpha, \beta \mid \mathbf{y}, \mathbf{x})$ on each point in the (α, β) -grid. Importantly, here we only need to evaluate the posterior **up to a normalization constant**, and hence, **we can evaluate the joint distribution instead of the posterior**. Moreover, for numerical reasons, we often evaluate the **logarithm of the joint distribution**.

Recall **Bayes' rule**:

$$p(\alpha, \beta \mid \mathbf{y}, \mathbf{x}) = \frac{p(\mathbf{y} \mid \mathbf{x}, \alpha, \beta) \cdot p(\alpha, \beta)}{p(\mathbf{y} \mid \mathbf{x})}$$

Since the denominator (evidence) is **constant w.r.t. α and β** , we only need to evaluate the **unnormalized posterior**, i.e., the joint distribution:

$$\tilde{\pi}_{ij} := p(\mathbf{y}, \alpha_i, \beta_j \mid \mathbf{x}) = p(\mathbf{y} \mid \mathbf{x}, \alpha_i, \beta_j) \cdot p(\alpha_i, \beta_j)$$

To improve **numerical stability**, we work in **log-space**:

$$\log \tilde{\pi}_{ij} = \log p(\mathbf{y} \mid \mathbf{x}, \alpha_i, \beta_j) + \log p(\alpha_i, \beta_j)$$

This is evaluated using the function provided to `Grid2D`, resulting in a matrix `log_joint_grid`.

We shift this matrix by subtracting the maximum value before exponentiating to prevent numerical overflow:

```
self.log_joint_grid = self.log_joint_grid - jnp.max(self.log_joint_grid)
self.tilde_probabilities_grid = jnp.exp(self.log_joint_grid)
```

3. **Normalization:** To ensure that our approximation is a valid distribution, we need to make it sum to 1. To achieve this, we compute the sum of $p(\mathbf{y}, \alpha, \beta \mid \mathbf{x})$ across all grid points to approximate the normalization constant Z .

That is,

$$q(\alpha_i, \beta_j) = \frac{1}{Z} p(\backslash \textcolor{red}{b}\textbf{m}y, \alpha_i, \beta_j | \mathbf{x}) = \frac{1}{Z} \tilde{\pi}_{ij} = \pi_{ij},$$

where we have defined $\log \tilde{\pi}_{ij} = \log p(\backslash \textcolor{red}{b}\textbf{m}y, \alpha_i, \beta_j | \mathbf{x})$ and $\pi_{ij} = \frac{1}{Z} \tilde{\pi}_{ij}$ for each point on the grid (α_i, β_j) . The normalization constant then becomes

$$Z = \sum_{i,j} p(\mathbf{y}, \alpha_i, \beta_j | \mathbf{x}) = \sum_{i,j} \tilde{\pi}_{ij} = \sum_{i=1}^I \sum_{j=1}^J \tilde{\pi}_{ij},$$

where the sum is over all the points in the grid. Note that $q(\alpha, \beta) = 0$ when evaluated on any point not belonging to the pre-defined (α, β) -grid.

Then define the **normalized grid**:

$$\pi_{ij} := q(\alpha_i, \beta_j) = \frac{\tilde{\pi}_{ij}}{Z}$$

This ensures that the grid approximation **sums to 1**:

$$\sum_{i,j} \pi_{ij} = 1$$

In code:

```
self.Z = jnp.sum(self.tilde_probabilities_grid)
self.probabilities_grid = self.tilde_probabilities_grid / self.Z
Finally, we flatten the grid to simplify later operations like sampling and computing expectations.
```

What Can We Do with the Grid Approximation?

With the grid-based approximation $q(\alpha, \beta) = \pi_{ij}$ in place, we can:

1. **Compute posterior expectations:**

$$\mathbb{E}_q[f(\alpha, \beta)] = \sum_{i,j} f(\alpha_i, \beta_j) \cdot \pi_{ij}$$

This is implemented in the `compute_expectation` method.

2. **Sample from the posterior:**

By interpreting π_{ij} as a probability mass function, we can generate samples from the posterior over (α, β) . This allows us to propagate parameter uncertainty to prediction uncertainty. See the `sample()` method.

3. **Visualize the posterior:**

Posterior modes, spread, and correlations between α and β can be visualized easily using `visualize()` .

GridApproximation2D Class

This class extends `Grid2D` and implements the full grid approximation procedure outlined above.

The relevant steps are:

- Compute the log-joint over the grid
- Exponentiate and normalize to get π_{ij}
- Store the flattened values for easy use
- Provide utility methods for sampling, visualization, expectations

It also includes sanity checks to verify that the grid sums to one and contains only valid probabilities.

As we will see later, the grid approximation makes it easy for us to

1. compute expectations with respect to the posterior
2. generate samples from the posterior, which will allow us to make inferences about the parameters α and β as well as make predictions will taking the uncertainty into account.

The purpose of the class `GridApproximation2D` given below is to approximate the posterior distribution of the logistic regression model using a grid approximation. We will re-use the code from the `Grid2D`-class to implemented step 1) and step 2) above. Hence, we basically also need to carry out the step 3.

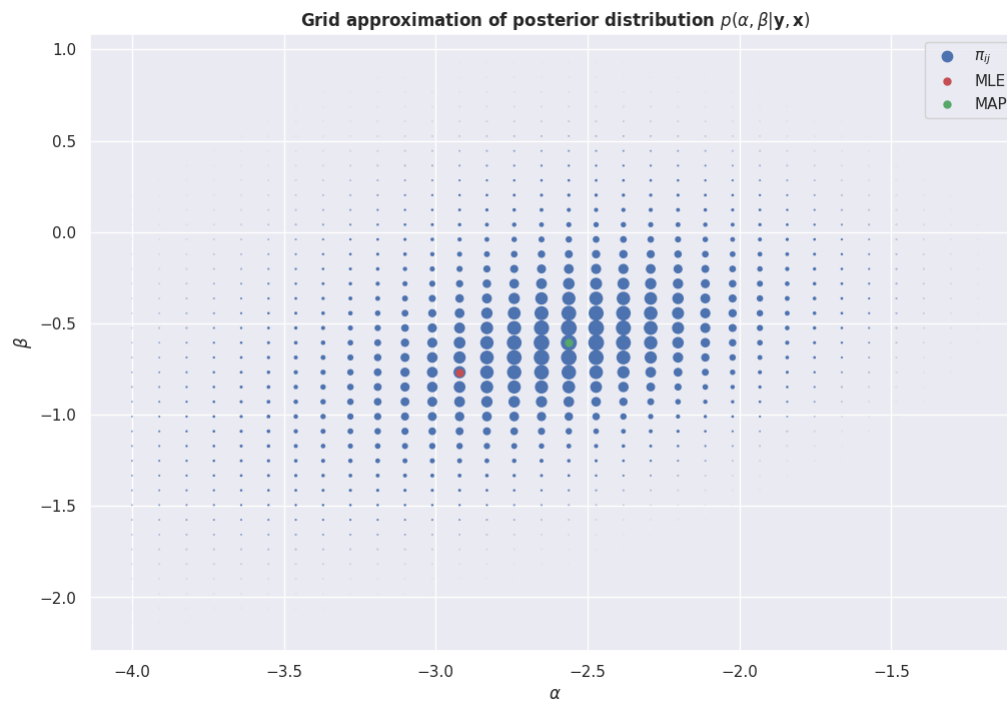
Task 4.1: Complete the implementation of the `prep_approximation`-function below. Your task is to make sure that the `probability_grid`-variable contains all the π_{ij} defined above with the proper normalization.

```
In [15]: # grid
num_alpha, num_beta = 90, 100
alphas = jnp.linspace(-4, 4, num_alpha)
betas = jnp.linspace(-4, 4, num_beta)

# model
model = LR.LogisticRegression(x, y, N, sigma2_alpha=1.0, sigma2_beta=1.0)

# construct grid approximation for posterior
post_approx = LR.GridApproximation2D(alphas, betas, model.log_joint, name='Posterior')

# plot
fig, ax = plt.subplots(1, 1, figsize=(12, 8))
post_approx.visualize(ax, title='Grid approximation of posterior distribution $p(\alpha, \beta | \mathbf{y}, \mathbf{x})$')
ax.plot(alpha_MLE, beta_MLE, 'r.', label='MLE', markersize=10);
ax.plot(alpha_MAP, beta_MAP, 'g.', label='MAP', markersize=10);
ax.legend();
```



 **Class: GridApproximation2D** (inherits from `Grid2D`)

 *What it does:*

This class turns a **log-joint probability function** into a **discrete probability distribution over a 2D grid** of parameters (α, β) .

You can:

-  Normalize the posterior over a grid
-  Sample from the posterior
-  Compute expectations under the approximate posterior
-  Visualize the distribution

This is a **grid-based approximation** to Bayesian inference — conceptually:

$$p(\alpha, \beta \mid \text{data}) \approx \frac{e^{\log p(\alpha, \beta, \text{data})}}{Z}$$

 Internals — Line by Line Walkthrough

class GridApproximation2D(Grid2D):

This class **inherits everything from Grid2D**, and adds:

- Proper normalization
- Sampling
- Expectation computation

def __init__(self, alphas, betas, log_joint, threshold=1e-8, name="GridApproximation2D"):

- alphas, betas: grid values
- log_joint: unnormalized posterior $\log p(\alpha, \beta, y)$
- threshold: used to filter very small probabilities for visualization
- Calls Grid2D.**init()** to compute self.values, i.e. the log-joint at each point

Then it calls:

```
self.prep_approximation()  
self.compute_marginals()  
self.sanity_check()
```

prep_approximation() : Core approximation logic

self.log_joint_grid = self.values
self.values is inherited from Grid2D and stores $\log p(\alpha, \beta, y)$

self.log_joint_grid = self.log_joint_grid - jnp.max(self.log_joint_grid)
 Numerical stability trick: subtract max before exponentiating, so you don't overflow

self.tilde_probabilities_grid = jnp.exp(self.log_joint_grid)
 Now you have unnormalized probabilities (i.e., $e^{\log p(\alpha, \beta, y)}$)

self.Z = jnp.sum(self.tilde_probabilities_grid)
self.probabilities_grid = self.tilde_probabilities_grid / self.Z
 This gives you a normalized posterior over the grid:

$$\pi_{ij} = \frac{p(\alpha_i, \beta_j, y)}{Z} = \frac{e^{\log p(\alpha_i, \beta_j, y)}}{\sum_{ij} e^{\log p(\alpha_i, \beta_j, y)}}$$

```
# flattening for sampling and expectations  
self.alphas_flat = self.alpha_grid.flatten()  
self.betas_flat = self.beta_grid.flatten()  
self.probabilities_flat = self.probabilities_grid.flatten()  
self.num_outcomes = len(self.alphas_flat)
```

These are useful for:

- vectorized expectations
- sampling

compute_marginals()

Computes marginals by summing across one axis:

```
self.pi_alpha = self.probabilities_grid.sum(1) # over beta  
self.pi_beta = self.probabilities_grid.sum(0)  # over alpha  
So you get
```

$$\pi_\alpha(\alpha) = \sum_\beta \pi(\alpha, \beta)$$

and

$$\pi_\beta(\beta) = \sum_\alpha \pi(\alpha, \beta)$$

```
compute_expectation(f)
```

Computer:

$$\mathbb{E}_q[f(\alpha, \beta)] = \sum_{i,j} f(\alpha_i, \beta_j) \cdot \pi_{ij}$$

In code:

```
def compute_expectation(self, f):  
    return jnp.sum(f(self.alphas_flat, self.betas_flat) * self.proBABILITIES_flat)
```

sample(key, num_samples) Draws samples from $\pi(\alpha, \beta)$ using JAX's random.choice :

```
idx = random.choice(key, jnp.arange(self.num_outcomes), p=self.proBABILITIES_flat, shape=(num_samples, 1))  
return self.alphas_flat[idx], self.betas_flat[idx]
```

You can use these samples to:

- simulate predictions
- compute predictive uncertainty
- visualize samples in parameter space

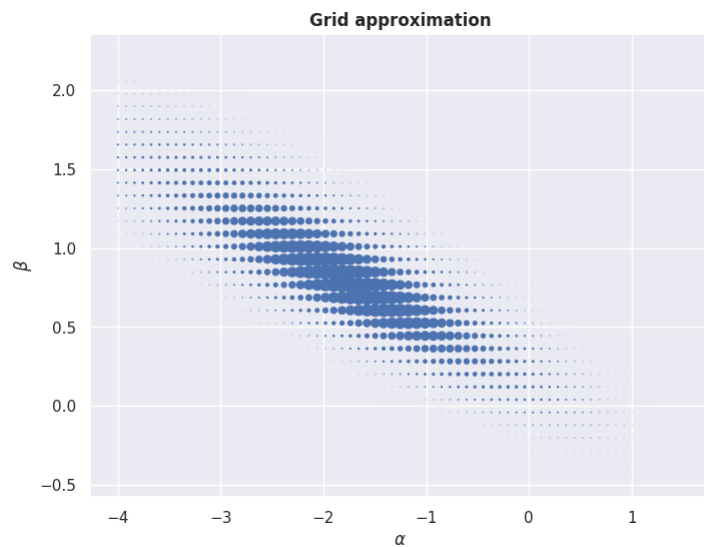
visualize(ax, scaling, title) Scatter plot:

- x-axis: α
- y-axis: β
- size of dot = posterior mass $\pi(\alpha, \beta)$

Filters out tiny probabilities (less than threshold).

✔ How to Use It: Full Example

```
In [27]: # Step 1: Define grid  
  
alphas_g2 = jnp.linspace(-4, 4, 100)  
betas_g2 = jnp.linspace(-4, 4, 100)  
  
# Step 2: Use log_joint from model  
log_joint_func_g2 = model.log_joint  
  
# Step 3: Create grid approximation  
grid_approx_g2 = LR.GridApproximation2D(alphas=alphas_g2, betas=betas_g2, log_joint=log_joint_func_g2)  
  
# Step 4: Visualize  
  
fig, ax = plt.subplots(figsize=(8, 6))  
grid_approx_g2.visualize(ax)  
plt.show()
```



What Can You Do With This?

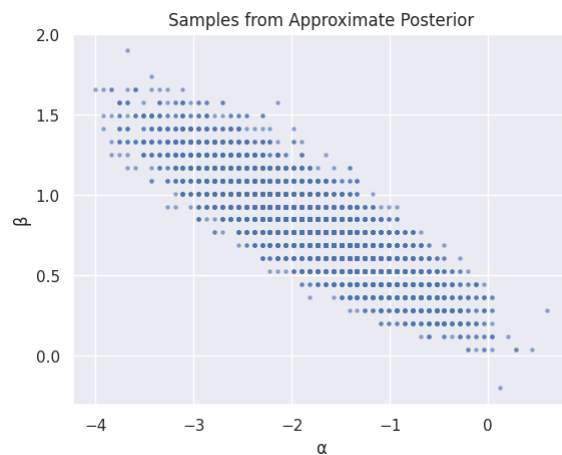
1. Plot Posterior Samples

```
In [28]: key = random.PRNGKey(0)

alpha_samples_g2, beta_samples_g2 = grid_approx_g2.sample(key, 10000)

plt.scatter(alpha_samples_g2, beta_samples_g2, s=5, alpha=0.5)
plt.xlabel("α")
plt.ylabel("β")
plt.title("Samples from Approximate Posterior")
```

Out[28]: Text(0.5, 1.0, 'Samples from Approximate Posterior')



2. Compute Expectations

```
In [29]: # Expected alpha
E_alpha = grid_approx_g2.compute_expectation(lambda a, b: a)
```

```
# Expected beta
E_beta = grid_approx_g2.compute_expectation(lambda a, b: b)
# Expected alpha * beta
E_ab = grid_approx_g2.compute_expectation(lambda a, b: a * b)

print(f"E_alpha = {E_alpha}")
print(f"E_beta = {E_beta}")
print(f"E_alpha * E_beta = {E_ab}")

E_alpha = -1.8003709316253662
E_beta = 0.8123219013214111
E_alpha * E_beta = -1.6048974990844727
```

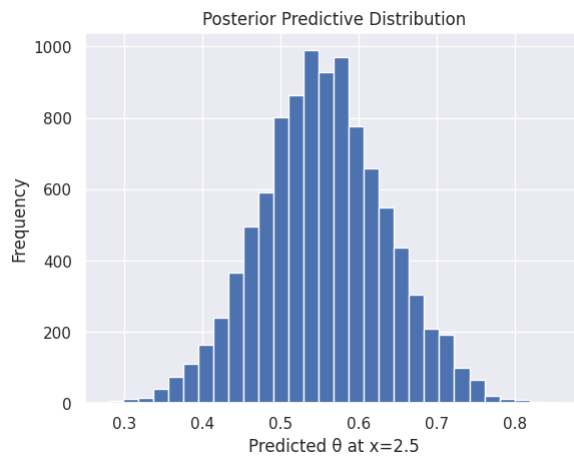
3. Predictive Distribution

Use samples to get predictive uncertainty at a new point:

```
In [30]: x_new = 2.5
theta_samples = model.theta(x_new, alpha_samples_g2, beta_samples_g2)

plt.hist(theta_samples, bins=30)
plt.xlabel("Predicted θ at x=2.5")
plt.ylabel("Frequency")
plt.title("Posterior Predictive Distribution")
```

```
Out[30]: Text(0.5, 1.0, 'Posterior Predictive Distribution')
```



Summary Table

Method	Purpose
<code>prep_approximation()</code>	Normalizes the unnormalized posterior
<code>compute_marginals()</code>	Gets marginal distributions over α and β
<code>compute_expectation(f)</code>	Computes expectations like $E[\alpha]$, $E[\beta]$, $E[\alpha\beta]$, etc.
<code>sample()</code>	Draws samples from posterior
<code>visualize()</code>	Visualizes the posterior grid
<code>sanity_check()</code>	Verifies correctness of the posterior

If the previous task was implemented correctly, the code above should produce a visualization of the grid approximation. The blue dots show the position of all the grid points, and the radius of the blue dots is controlled by the size of the corresponding weight π_{ij} . For easier interpretation, we do not plot points where π_{ij} is smaller than 10^{-8} .

Our first order of business after having obtained the posterior distribution is to produce estimates of the parameters α and β as well to quantify the uncertainty, i.e. we will estimate the posterior mean, posterior standard deviation and 95% credibility interval for both α and β . For this purpose, it is more convenient to work with the marginal posterior distributions of α and β , which can be readily obtained from the joint posterior approximation $q(\alpha, \beta)$ using the **sum rule**:

$$q(\alpha_i) = \sum_j q(\alpha_i, \beta_j) = \sum_j \pi_{ij} = \pi_i^\alpha$$

$$q(\beta_j) = \sum_i q(\alpha_i, \beta_j) = \sum_i \pi_{ij} = \pi_j^\beta,$$

i.e. $\pi^\alpha = q(\alpha_i)$ is the (approximate) marginal posterior probability of $p(\alpha = \alpha_i | \mathbf{y}, \mathbf{x})$.

Task 4.2: Go back up the class `GridApproximation2D` and implemented the `compute_marginals`-function. Use it to compute and plot the probability mass functions for the approximate posterior marginal distributions $q(\alpha)$ and $q(\beta)$ as a function of α and β , respectively.

Task 4.3: Compute the posterior mean and standard deviation for α and β using the grid approximation by completing the implementation of `DiscreteDistribution1D` -class below. Use the figures from the previous task to verify that your results are (approximately) correct.

Hints:

- The mean of a discrete random variable X with PMF $P(X = x)$ is given by $\mathbb{E}[X] = \sum_i x_i P(X = x_i)$, where the sum is over all the possible outcomes of X .
- The variance of a discrete random variable X with PMF $P(X = x)$ is given by $\mathbb{V}[X] = \sum_i (x_i - \mathbb{E}[X])^2 P(X = x_i)$, where the sum is over all the possible outcomes of X .

```
In [16]: class DiscreteDistribution1D(object):

    def __init__(self, outcomes, probabilities, name='DiscreteDistribution'):
        """ represents discrete random variable X in terms of outcomes and probabilities """
        self.outcomes = outcomes
        self.probabilities = probabilities
        assert self.outcomes.shape == self.probabilities.shape
        self.name = name

    def CDF(self, x):
        """ P[X <= x] """
        idx = self.outcomes <= x
        return jnp.sum(self.probabilities[idx])

    def quantile(self, p):
        """ Q(p) = inf {x | p < CDF(x)} """
        cdf_values = jnp.cumsum(self.probabilities)
        idx = jnp.where(jnp.logical_or(p < cdf_values, jnp.isclose(p, cdf_values)))[0]
        return jnp.min(self.outcomes[idx])

    @property
    def mean(self):
        """ return scalar corresponding to the mean of the discrete distribution """
        return jnp.mean(self.outcomes)

    @property
    def variance(self):
        """ return scalar corresponding to the variance of the discrete distribution """
        return jnp.sum((self.outcomes - self.mean)**2 * self.probabilities)

    def central_interval(self, interval_size=95):
        """ return tuple (lower, upper) corresponding to the central interval of the discrete distribution """
        c = 1.-interval_size/100.
        lower = self.quantile(c/2)
        upper = self.quantile(1-c/2)
        return jnp.array([lower, upper])

    def print_summary(self):
        print(f'Summary for {self.name}')
        print(f'\tMean:\t\t\t\t{self.mean:3.2f}')
        print(f'\tStd. dev.:\t\t\t\t{jnp.sqrt(self.variance):3.2f}')
        print(f'\t95%-credibility interval:\t\t[{self.central_interval()[0]:3.2f}, {self.central_interval()[1]:3.2f}]\n')

# example use and sanity checks using classic six-sided die
outcomes, probs = jnp.arange(1,6+1), jnp.ones(6)/6
sixsided_die = DiscreteDistribution1D(outcomes, probs)
assert jnp.allclose(sixsided_die.mean, 3.5), f"The mean of a sixsided die should be 3.5, but the value was {sixsided_die.mean:3.2f}. Check your implementation."
assert jnp.allclose(sixsided_die.variance, 35/12), f"The variance of a sixsided die should be approximately 2.92, but the value was {sixsided_die.variance:3.2f}. Check your implementation."
assert jnp.allclose(sixsided_die.central_interval(95), jnp.array([1, 6])), f"The (approximate) 95% central interval of a sixsided die should be (1,6), but the value was {sixsided_die.central_interval(95)}. Check your implementation."

# summarize posterior marginals for alpha and beta
post_alpha_marginal = DiscreteDistribution1D(post_approx.alphas, post_approx.pi_alpha, "posterior distribution of alpha p(alpha|D)")
post_beta_marginal = DiscreteDistribution1D(post_approx.betas, post_approx.pi_beta, "posterior distribution of beta p(beta|D)")

post_alpha_marginal.print_summary()
post_beta_marginal.print_summary()
```

```
Summary for posterior distribution of alpha p(alpha|D)
Mean: 0.00
Std. dev.: 2.63
95%-credibility interval: [-3.28, -2.02]
```

```
Summary for posterior distribution of beta p(beta|D)
Mean: 0.00
Std. dev.: 0.66
95%-credibility interval: [-1.17, -0.04]
```

Compare the summary statistics to the plots above. Verify that your results are consistent before moving on.

Task 4.4: Compute the prior and posterior probability that $\beta < 0$

```
In [17]: print(f"Prior probability of p(Beta < 0): {jnp.mean(betas < 0)}")
print(f"Posterior probability of p(Beta < 0): {jnp.mean(post_approx.pi_beta < 0)}")
```

```
Prior probability of p(Beta < 0): 0.5
Posterior probability of p(Beta < 0): 0.0
```

 **Class: DiscreteDistribution1D**

 *Purpose:*

Represents a discrete 1D probability distribution such as:

- Posterior marginal of α or β
- Any probability mass function (PMF) over finite outcomes

You can:

- Compute the CDF: $P(X \leq x)$
- Compute quantiles and credible intervals
- Get the mean and variance
- Print a summary (mean, std, and 95% interval)

 **Method-by-method Breakdown**

```
__init__(self, outcomes, probabilities, name="...")
Initializes the object:
```

- outcomes: 1D array (e.g., possible values of α or β)
- probabilities: 1D array of same shape, summing to 1
- name: string used in summaries and debugging

```
assert self.outcomes.shape == self.probabilities.shape
Ensures valid distribution.
```

CDF(self, x)

Return the cumulative distribution function (CDF) at point x:

$$\text{CDF}(x) = P(X \leq x) = \sum_{i: x_i \leq x} P(X = x_i) = \sum_{i: x_i \leq x} \pi[i]$$

Used to find probabilities up to a threshold.

```
idx = self.outcomes <= x
return jnp.sum(self.probabilities[idx])
```

quantile(self, p) Computes:

$$Q(p) = \inf\{x : P(X \leq x) \geq p\}$$

Equivalent to inverse CDF or percentile function.

```
cdf_values = jnp.cumsum(self.probabilities)
idx = jnp.where(jnp.logical_or(p < cdf_values, jnp.isclose(p, cdf_values)))[0]
return jnp.min(self.outcomes[idx])
```

mean (property) Computes

$$\mathbb{E}[X] = \sum_i x_i P(X = x_i)$$

return jnp.mean(self.outcomes)

⚠ This assumes uniform weights. For arbitrary probabilities, you'd use:

return jnp.sum(self.outcomes * self.probabilities)

variance (property)

Computes:

$$\text{Var}[X] = \sum_i (x_i - \mathbb{X})^2 \cdot p_i$$

return jnp.sum((self.outcomes - self.mean)**2 * self.probabilities)

central_interval(interval_size=95)

Computes the central credible interval of size `interval_size` %:

```
c = 1 - interval_size / 100
lower = self.quantile(c / 2)
upper = self.quantile(1 - c / 2)
return jnp.array([lower, upper])
```

print_summary()

Nicely formatted terminal output:

```
print(f'Summary for {self.name}')
print(f'\tMean:\t\t\t{self.mean:3.2f}')
print(f'\tStd. dev.:\t\t\t{jnp.sqrt(self.variance):3.2f}')
print(f'\t95%-credibility interval:\t[{self.central_interval()[0]:3.2f}, {self.central_interval()[1]:3.2f}]\n')
```

🧠 What the GridApproximation2D Class Is Doing (Mathematically)

This class helps implement:

$$p(\alpha, \beta \mid \mathbf{y}, \mathbf{x}) \propto p(\mathbf{y} \mid \alpha, \beta, \mathbf{x}) \cdot p(\alpha, \beta)$$

This is **Bayes' rule** in the context of **Bayesian logistic regression**:

- **Posterior:** $p(\alpha, \beta \mid \mathbf{y}, \mathbf{x})$
- **Likelihood:** $p(\mathbf{y} \mid \alpha, \beta, \mathbf{x})$
- **Prior:** $p(\alpha, \beta)$

But since logistic regression **does not have a conjugate posterior**, we **cannot compute this posterior exactly**.

Instead, `GridApproximation2D` discretizes the posterior on a 2D grid of (α, β) values and normalizes it into a valid probability distribution.

💡 How `GridApproximation2D` Implements the Equation

- ♦ 1. Create the Unnormalized Posterior on a Grid

You provide:

- A grid of values for α and β (e.g., using `jnp.linspace`)
- A log-joint function:

$$\log p(\alpha, \beta, \mathbf{y}) = \log p(\mathbf{y} \mid \alpha, \beta, \mathbf{x}) + \log p(\alpha, \beta)$$

This is passed to the constructor:

```
post_approx = GridApproximation2D(alphas, betas, model.log_joint)
which internally computes:
```

```
self.values = model.log_joint(self.alpha_grid, self.beta_grid)
```

So at each grid point (α_i, β_j) :

```
self.values[i, j] = \log p(\mathbf{y} \mid \alpha_i, \beta_j) + \log p(\alpha_i, \beta_j)
```

2. Convert Log-Joint to Normalized Posterior

Inside `prep_approximation()` :

```
self.tilde_probabilities_grid = jnp.exp(self.log_joint_grid)
self.Z = jnp.sum(self.tilde_probabilities_grid)
self.probabilities_grid = self.tilde_probabilities_grid / self.Z
```

This computes:

$$\pi_{ij} = \frac{e^{\log p(\mathbf{y}|\alpha_i, \beta_j) + \log p(\alpha_i, \beta_j)}}{Z}$$

Each π_{ij} is your **posterior mass** at (α_i, β_j) .

How to Interpret the Posterior Plot

```
fig, ax = plt.subplots(1, 1, figsize=(12, 8))
post_approx.visualize(ax, title='Grid approximation of posterior distribution')
```

This shows:

- Each dot = a pair (α, β) on the grid
- Dot size = posterior mass π_{ij}
- You'll see a "cloud" around high-density regions

Understanding the MLE and MAP Overlay

```
ax.plot(alpha_MLE, beta_MLE, 'r.', label='MLE')
ax.plot(alpha_MAP, beta_MAP, 'g.', label='MAP')
```

- `alpha_MLE` , `beta_MLE` : obtained from `log_lik_grid.argmax`
- `alpha_MAP` , `beta_MAP` : obtained from `post_approx.argmax`

Since `GridApproximation2D` inherits from `Grid2D` , it exposes `.argmax` to get MAP.

Summary — Putting It All Together

Concept	Math	Code
Grid of parameters	(α_i, β_j)	<code>jnp.meshgrid(alphas, betas)</code>
Unnormalized posterior	$p(\mathbf{y} \mid \alpha, \beta) \cdot p(\alpha, \beta)$	<code>self.log_joint_grid</code>
Normalized posterior	$\pi_{ij} = \frac{e^{\log(\cdot)}}{Z}$	<code>self.probabilities_grid</code>
Posterior plot	Point size = π_{ij}	<code>visualize()</code>
MAP	$\arg \max_{\alpha, \beta} p(\alpha, \beta, \mathbf{y})$	<code>post_approx.argmax</code>
MLE	$\arg \max_{\alpha} p(\mathbf{y} \mid \alpha, \beta)$	<code>log_lik_grid.argmax</code>

What Are Posterior Marginals?

Let's start from the **joint posterior**:

$$p(\alpha, \beta \mid \mathbf{y}, \mathbf{x})$$

This is a **2D distribution** over both α and β , computed via Bayes' rule:

$$p(\alpha, \beta \mid \mathbf{y}, \mathbf{x}) \propto p(\mathbf{y} \mid \alpha, \beta) \cdot p(\alpha, \beta)$$

♦ Posterior Marginal of α

This is the distribution of α **alone**, obtained by **summing out** all possible values of β :

$$p(\alpha \mid \mathbf{y}, \mathbf{x}) = \sum_{\beta} p(\alpha, \beta \mid \mathbf{y}, \mathbf{x})$$

This process is called **marginalization**:

▮ "What is the probability of each value of α , regardless of what β is?"

♦ Posterior Marginal of β

Likewise:

$$p(\beta \mid \mathbf{y}, \mathbf{x}) = \sum_{\alpha} p(\alpha, \beta \mid \mathbf{y}, \mathbf{x})$$

🧠 Why Do We Care?

Marginal distributions let us:

- Understand **uncertainty in each parameter separately**
- Compute **credible intervals** (e.g., " α is likely between -2.5 and -0.5")
- Summarize the posterior with **mean and standard deviation**
- Make **marginal decisions** (e.g., fix one parameter, reason about the other)

✅ In Your Code: Where This Happens

Inside `GridApproximation2D`, marginals are computed as:

```
def compute_marginals(self):
    self.pi_alpha = self.probabilities_grid.sum(1) # Sum over beta (columns)
    self.pi_beta = self.probabilities_grid.sum(0) # Sum over alpha (rows)
```

- `self.probabilities_grid`: 2D posterior over (α, β)
- `self.pi_alpha`: marginal over α
- `self.pi_beta`: marginal over β

📊 How to Use Posterior Marginals

Use the `DiscreteDistribution1D` class:

```
post_alpha_marginal = DiscreteDistribution1D(
    outcomes=post_approx.alphas,
    probabilities=post_approx.pi_alpha,
    name="Posterior of alpha"
)
```

```
post_beta_marginal = DiscreteDistribution1D(
    outcomes=post_approx.betas,
    probabilities=post_approx.pi_beta,
    name="Posterior of beta"
)
```

```
post_alpha_marginal.print_summary()
post_beta_marginal.print_summary()
```

This will give you:

- Posterior **mean**
- Posterior **standard deviation**
- **95% credible intervals**

Visual Summary

Joint Posterior: $p(\alpha, \beta \mid \text{data})$



→ Marginal over β (sum rows)

↓ Marginal over α (sum columns)

In Words

- **Marginal of α** answers:
"What do I believe about α , regardless of β ?"
- **Marginal of β** answers:
"What do I believe about β , regardless of α ?"

Part 5: Propagating uncertainty from parameters to predictions

Using the grid approximation, we got a handle on the posterior distribution of α and β given the observed data, i.e. $p(\alpha, \beta \mid \mathbf{y})$. We will now turn our attention to making predictions. For example, we want to predict the following quantities for a given temperature $t^* \in \mathbb{R}$, (standardized to x^*):

- The probability of failure for each o-ring
- The probability of at least one o-ring failing
- The expected number of failing o-rings
- A 95%-credibility interval for then number of failed o-rings

Assuming we know the value of α and β , all quantities above are easily computed from the predictive likelihood

$$p(y^* \mid x^*, \alpha, \beta) = \text{Bin}(y^* \mid N^*, \alpha, \beta), \quad \text{where } \theta^* = \sigma(\alpha + \beta x^*). \quad (12)$$

This give us the full distribution of possible failure counts y^* for the given x^* .

However, in practice we do not now the value of α and β , and therefore, we need to estimate them. The simplest approach for making predictions is to use a **plugin approximation** like we did in part 3. That is, we simply plug in one of our point estimates for α and β into the equation above, e.g. the MLE or MAP estimator. This is usually fast and easy to do, but the significant drawback is that it tends to produce **overconfident** predictions because it ignores the uncertainty for θ^* . Instead, we can propagate the uncertainty from the posterior distribution $q(\alpha, \beta)$ to the predictive distribution using **marginalization** via the sum:

$$p(y^* \mid \mathbf{y}, \mathbf{x}, x^*) = \int \int p(y^* \mid x^*, \alpha, \beta) p(\alpha, \beta \mid \mathbf{y}, \mathbf{x}) d\alpha d\beta = \mathbb{E}_{p(\alpha, \beta \mid \mathbf{y}, \mathbf{x})} [p(y^* \mid x^*, \alpha, \beta)] \approx \mathbb{E}_{q(\alpha, \beta)} [p(y^* \mid x^*, \alpha, \beta)]. \quad (13)$$

That is, when we make predictions, we average over the posterior distribution. **Instead of picking a single set of parameters, we consider all possible values for α and β and weigh them according to their posterior probability.** This may be abstract, but the grid approximation can help understand this more intuitively:

$$p(y^* \mid \mathbf{y}, \mathbf{x}, x^*) = \mathbb{E}_{p(\alpha, \beta \mid \mathbf{y})} [p(y^* \mid x^*, \alpha, \beta)] \approx \mathbb{E}_{q(\alpha, \beta)} [p(y^* \mid x^*, \alpha, \beta)] = \sum_{ij} p(y^* \mid x^*, \alpha_i, \beta_j) \pi_{ij}, \quad (14)$$

where we literally evaluate $p(y^* \mid x^*, \alpha, \beta)$ for each possible set of parameters (α_i, β_j) and weigh them by the posterior probability $\pi_{ij} = q(\alpha_i, \beta_j)$. We refer to the distribution $p(y^* \mid \mathbf{y}, \mathbf{x}, x^*)$ as the **posterior predictive distribution**. Recall, we can also cast the plugin approximations as approximation over the posterior predictive distribution, where the posterior is approximated by a posterior distribution that as all the mass on the single point $(\hat{\alpha}_{MLE}, \hat{\beta}_{MLE})$.

The integral in (2) may not be analytically tractable, but with the grid approximation, we can compute this expectation numerically, as given in the sum above in equation (2). Where:

(α_i, β_j) is a point on the grid,

$\pi_{ij} \approx p(\alpha_i, \beta_j \mid \mathbf{x}, \mathbf{y})$ is the approximate posterior probability at that grid point.

Thus, we evaluate the likelihood at each grid point and average the predictions, weighted by the posterior probabilities.

Summary of Key Idea Instead of selecting a single “best guess” for the parameters, we average over all plausible parameter values, weighted by how probable they are under the posterior. This produces predictions that are more calibrated and less overconfident. In this way, the posterior predictive distribution $p(y^* \mid \mathbf{x}, \mathbf{y}, x^*)$ reflects both the inherent uncertainty in future outcomes and the epistemic uncertainty about the parameters themselves.

The code below uses the plugin approximation for the MLE/MAP estimator to compute and plot the posterior predictive distribution $p(y^* = k \mid \mathbf{y}, \mathbf{x}, x^*)$ for $k \in \{0, 1, \dots, 6\}$. The code also reports the mean and a 95% interval for each methods for each temperatures (assuming the tasks above have been solved).

Task 5.1: Compute and plot the full posterior predictive distribution $p(y^* = k | \mathbf{y}, \mathbf{x}, x^*)$ with $k \in \{0, 1, \dots, 6\}$ for $t = 25^\circ C$ and $t = 0^\circ C$.

```
In [18]: # target temperatures t^*
temp_stars = [25, 0]

# outcome space for y^*
ks = jnp.arange(0, 6+1)

# predictive likelihood p(y^*=k|x^*, alpha, beta)
pred_lik = lambda k, x_star, alpha, beta: binom_dist.pmf(k=k, n=6, p=model.theta(x_star, alpha, beta))

# prep figure
fig, ax = plt.subplots(2, 3, figsize=(20, 10))

mean_and_interval_label = lambda dist: f'Mean: {dist.mean:3.2f}, 95%-interval: [{dist.central_interval()[0]:d}, {dist.central_interval()[1]:d}]'

for n, t_star in enumerate(temp_stars):

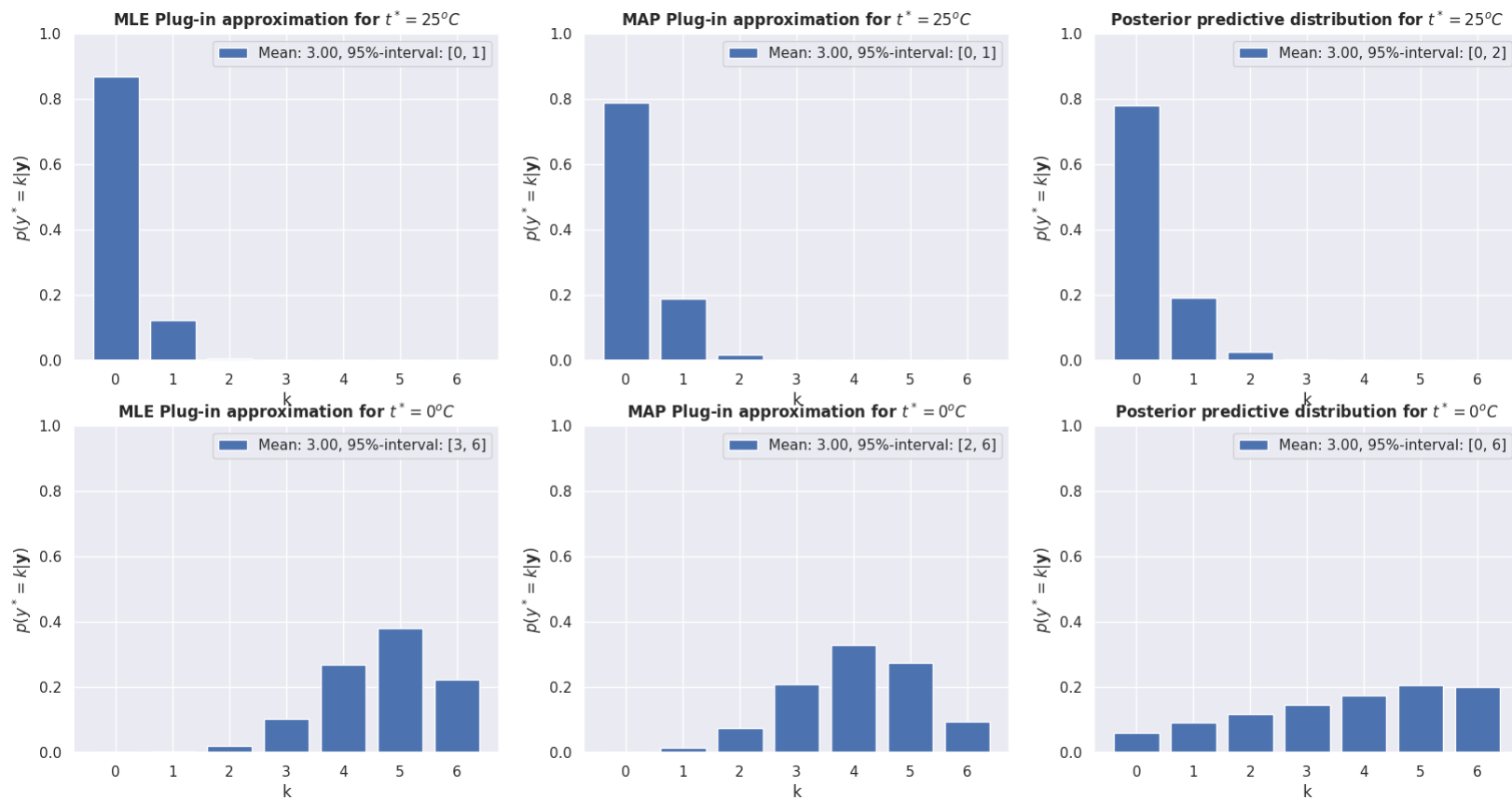
    # standardize to get x^*
    x_star = standardize(t_star)

    # compute plugin approximations using MLE and MAP. Shape must be [7] (matching the shape of ks)
    pred_lik_probs_MLE = pred_lik(ks, x_star, alpha_MLE, beta_MLE)
    pred_lik_probs_MAP = pred_lik(ks, x_star, alpha_MAP, beta_MAP)

    # compute posterior predictive distribution as a weighted averaged over all parameter vales. Shape must be [7] (matching the shape of ks)
    pred_lik_bayes_probs = jnp.sum(pred_lik(ks, x_star, post_approx.alphas_flat[:, None], post_approx.betas_flat[:, None])*post_approx.probabilities_flat[:, None], axis=0)

    # prepare for plotting etc
    pred_dist_MLE = DiscreteDistribution1D(ks, pred_lik_probs_MLE, name='MLE Plug-in approximation')
    pred_dist_MAP = DiscreteDistribution1D(ks, pred_lik_probs_MAP, name='MAP Plug-in approximation')
    pred_dist_bayes = DiscreteDistribution1D(ks, pred_lik_bayes_probs, name="Posterior predictive distribution")

    # plot
    for i, dist in enumerate([pred_dist_MLE, pred_dist_MAP, pred_dist_bayes]):
        ax[n, i].bar(ks, dist.probabilities, label=mean_and_interval_label(dist))
        ax[n, i].set(xlabel='k', ylabel='p(y^* = k|\\mathbf{y})$', ylim=(0, 1))
        ax[n, i].set_ylim((0, 1))
        ax[n, i].set_title(f'{dist.name} for $t^* = {t_star}^{\circ}C$', fontweight='bold')
        ax[n, i].set(xlabel='k', ylabel='p(y^* = k|\\mathbf{y})$', ylim=(0, 1))
        ax[n, i].legend()
```



Task 5.2: Comment on the differences between the results obtained using plugin approximations, i.e. the MLE/MAP estimators, and the posterior predictive distributions. How does the three approximations compare? In regions with data? In regions without data?

Note: *This is a discussion question, which means that you actively have to experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs.***

In the very last of the exercise, we will visualize the posterior for $f(x)$ and $\theta(x)$ to build over intuition. We will start by drawing a set of random samples of (α, β) from the posterior approximation and then compute and visualize $f(x)$ and $\theta(x)$ for each sample. We will repeat this process for the prior distribution $p(\alpha, \beta)$.

Task 5.3: The code below generates `num_samples` of α, β from both the prior and posterior and plots them. Compute $f(x_{\text{pred}})$ and $\theta(x_{\text{pred}})$ for each of these samples and store them in the variables `f_samples` and `theta_samples` pre-defined below.

```
In [19]: # data for predictions
temp_pred = jnp.linspace(-10, 40, 500)
x_pred = standardize(temp_pred)
N_pred = 6

# number of samples plot
num_samples = 100

# grid
num_alpha, num_beta = 90, 100
alphas = jnp.linspace(-4, 4, num_alpha)
betas = jnp.linspace(-4, 4, num_beta)

# set up prior and posterior for visualization purposes
model = LR.LogisticRegression(x, y, N, sigma2_alpha=1., sigma2_beta=1.)
post_approx = LR.GridApproximation2D(alphas, betas, model.log_joint, name='Posterior')
prior_approx = LR.GridApproximation2D(alphas, betas, model.log_prior, name='Prior')
```

```
# specify seed
seed = 123
key = random.PRNGKey(seed)

# plot for each distribution...
fig, ax = plt.subplots(2, 3, figsize=(20, 8))
for idx_plot, dist in enumerate([prior_approx, post_approx]):
    key, subkey = random.split(key)

    # plot data and contours of distribution
    plot_data(ax[idx_plot, 2], counts=False)
    dist.plot_contours(ax[idx_plot, 0], f=jnp.exp, color=colors[idx_plot])

    # get (alpha, beta)-samples to visualize
    alpha_samples, beta_samples = dist.sample(subkey, num_samples)

    # compute samples of f(x_pred) and theta(x_pred). Shape must be [num_samples, len(temp_pred)] for both f_samples and theta_samples
    f_samples = model.f(x_pred, alpha_samples, beta_samples)
    theta_samples = model.theta(x_pred, alpha_samples, beta_samples)

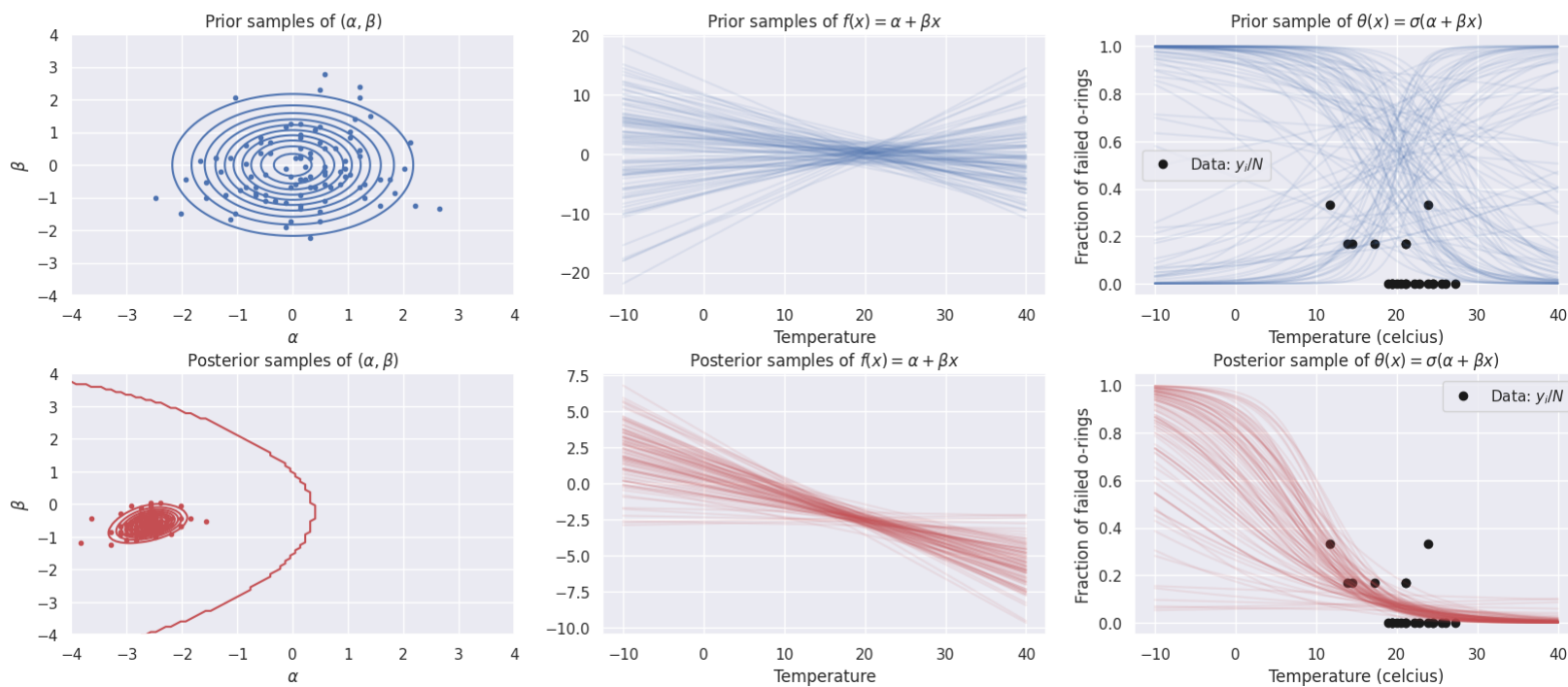
    # plot
    ax[idx_plot, 0].plot(alpha_samples, beta_samples, '.', color=colors[idx_plot])

    # plot each sample of f and theta
    ax[idx_plot, 1].plot(temp_pred, f_samples.T, '', color=colors[idx_plot], alpha=0.1)
    ax[idx_plot, 2].plot(temp_pred, theta_samples.T, '', color=colors[idx_plot], alpha=0.1)
    ax[idx_plot, 2].legend()

    # title and labels
    ax[idx_plot, 0].set(title=f'{dist.name} samples of $(\\alpha, \\beta)$')
    ax[idx_plot, 1].set(title=f'{dist.name} samples of $f(x) = \\alpha + \\beta x$, xlabel='Temperature')
    ax[idx_plot, 2].set(title=f'{dist.name} sample of $\\theta(x) = \\sigma(\\alpha + \\beta x)$')

fig.suptitle('Prior and posterior predictions', fontweight='bold')
fig.subplots_adjust(hspace=0.3)
```

Prior and posterior predictions



Task 5.4: Study the code and plots above and make sure you can explain the following:

- What is plotted in each of the six panels?
- What is the relationship between each of the dots in the left columns and the curves in the center and right columns?
- How the uncertainty on $f(x)$ and $\theta(x)$ is controlled by the distribution of (α, β) ?
- How well does each of the prior samples match the observed data?
- How well does each of the posterior samples match the observed data?

Note: *This is a discussion question, which means that you actively have to experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs.*

We can also use posterior samples to summarize the pointwise posterior mean and 95% intervals of $f(x)$ and $\theta(x)$:

```
In [20]: # set seed
seed = 0
key = random.PRNGKey(seed)

# set up prior and posterior for visualization purposes
model = LR.LogisticRegression(x, y, N, sigma2_alpha=1., sigma2_beta=1.)
post_approx = LR.GridApproximation2D(alphas, betas, model.log_joint, name='Posterior')
prior_approx = LR.GridApproximation2D(alphas, betas, model.log_prior, name='Prior')

# plot
fig, ax = plt.subplots(1, 2, figsize=(20, 6))
plot_data(ax[0], counts=False)
plot_data(ax[1], counts=False)

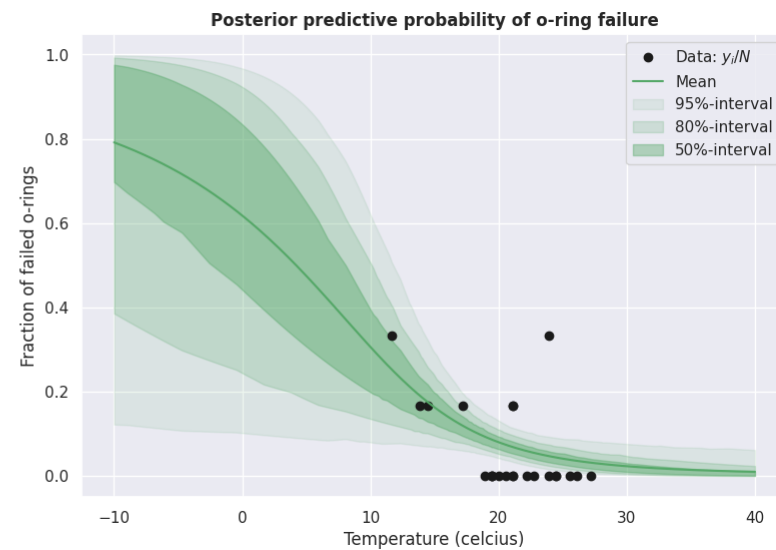
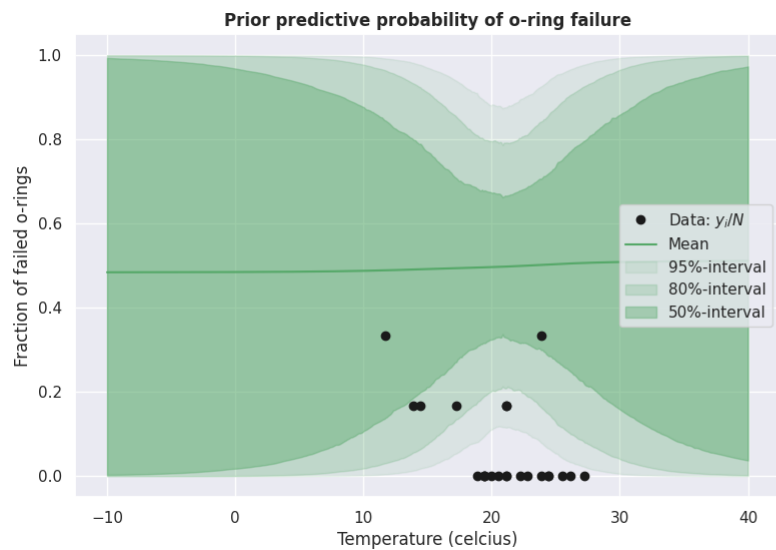
num_samples = 2000

for idx_plot, dist in enumerate([prior_approx, post_approx]):
    key, subkey = random.split(key)

    alpha_samples, beta_samples = dist.sample(subkey, num_samples)

    theta_samples = model.theta(x_pred, alpha_samples, beta_samples)
    theta_mean = theta_samples.mean(0)
    theta_lower = jnp.quantile(theta_samples, 0.025, axis=0)
    theta_upper = jnp.quantile(theta_samples, 1-0.025, axis=0)

    ax[idx_plot].plot(temp_pred, theta_mean, 'g-', label='Mean')
    for interval_size, b in [(0.95, 0.1), (0.8, 0.2), (0.5, 0.4)]:
        c = 1-interval_size
        theta_lower = jnp.quantile(theta_samples, c/2, axis=0)
        theta_upper = jnp.quantile(theta_samples, 1-c/2, axis=0)
        ax[idx_plot].fill_between(temp_pred, theta_lower, theta_upper, color='g', alpha=b, label=f'{interval_size*100:2.0f}%-interval')
    ax[idx_plot].legend()
    ax[idx_plot].set_title(f'{dist.name} predictive probability of o-ring failure', fontweight='bold')
```



Task 5.5: Compute and plot the posterior mean and a 95% credibility interval for the probability of obtaining at least one failure among the 6 o-rings

```
In [22]: # set seed and number of samples
seed = 0
key = random.PRNGKey(seed)
num_samples = 2000

# set up prior and posterior for visualization purposes
model = LR.LogisticRegression(x, y, N, sigma2_alpha=1., sigma2_beta=1.)
post_approx = LR.GridApproximation2D(alphas, betas, model.log_joint, name='Posterior')

alpha_samples, beta_samples = post_approx.sample(key, num_samples)
theta_samples = model.theta(x_pred, alpha_samples, beta_samples)

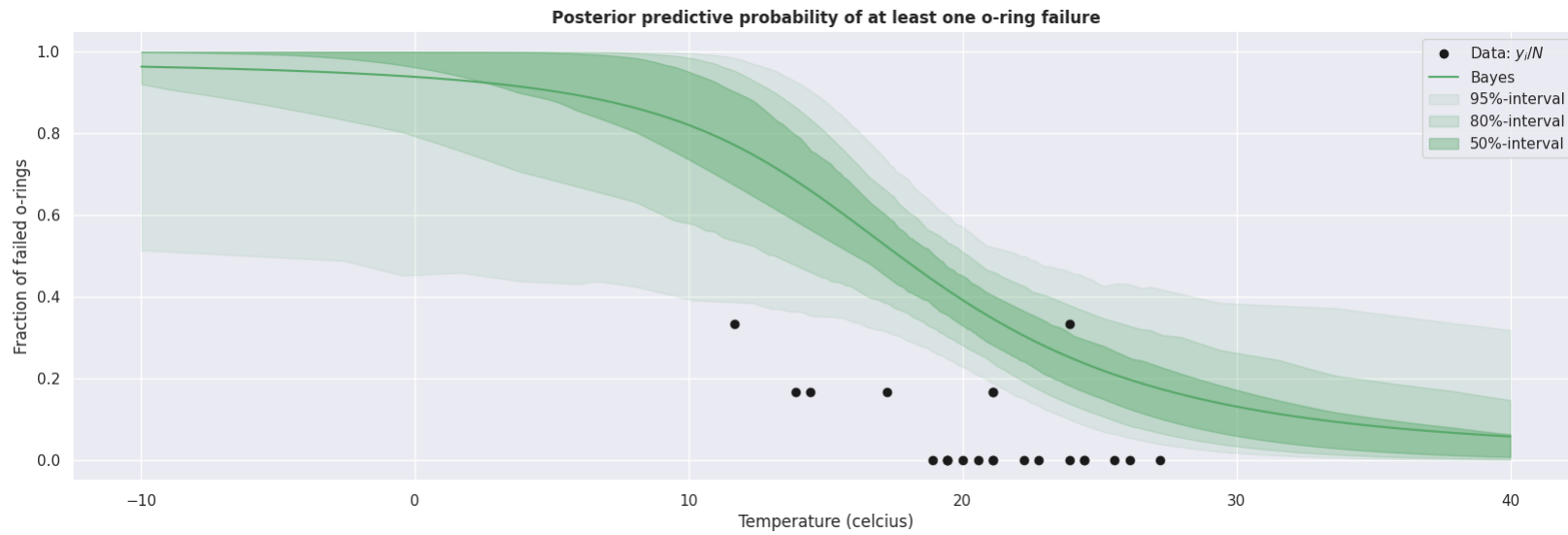
# probability of at least one failure wrt. a binomial post_approximation
prob_at_least_one_failure = 1 - (1 - theta_samples)**N_pred

# compute mean and interval
post_mean = prob_at_least_one_failure.mean(0)
post_lower = jnp.quantile(prob_at_least_one_failure, 0.025, axis=0)
post_upper = jnp.quantile(prob_at_least_one_failure, 1-0.025, axis=0)

# plot
fig, ax = plt.subplots(1, 1, figsize=(20, 6))
plot_data(ax, counts=False)

ax.plot(temp_pred, post_mean, 'g-', label='Bayes')
for interval_size, b in [(0.95, 0.1), (0.8, 0.2), (0.5, 0.4)]:
    c = 1 - interval_size
    post_lower = jnp.quantile(prob_at_least_one_failure, c/2, axis=0)
    post_upper = jnp.quantile(prob_at_least_one_failure, 1-c/2, axis=0)
    ax.fill_between(temp_pred, post_lower, post_upper, color='g', alpha=b, label=f'{interval_size*100:2.0f}%-interval')
ax.legend()
ax.set_title(f'{post_approx.name} predictive probability of at least one o-ring failure', fontweight='bold')

Out[22]: Text(0.5, 1.0, 'Posterior predictive probability of at least one o-ring failure')
```



In []:

In []:

In []: